

龙芯环境搭建软件包

通过网盘分享的文件：龙芯环境搭建软件

链接: <https://pan.baidu.com/s/1GUz4VZZBqmnnQcF4jpTl2Q?pwd=cu9e> 提取码: cu9e

[返回上一级](#) | [全部文件](#) > 龙芯环境搭建软件

☐	文件名	大小
☐	 Ubuntu 24	-
☐	 VMware-workstation-full-17.0.2-21581411.exe	607.7M
☐	 MobaXterm_Portable_v25.1.zip	41.3M

UBUNTU安装

省略一万个字...

Ubuntu 24.04更换清华源

1.备份原有的源配置文件：

打开终端，输入以下命令来备份原有的源配置文件，也可以直接不备份。

```
sudo cp /etc/apt/sources.list.d/ubuntu.sources
/etc/apt/sources.list.d/ubuntu.sources.bak
```

2.编辑源配置文件：

使用vi或其他文本编辑器打开源配置文件：

```
sudo vi /etc/apt/sources.list.d/ubuntu.sources
```

3. 添加清华源的配置：

在打开的文件中，删除原有的内容，并粘贴以下清华源的配置：

```
Types: deb
URIs: https://mirrors.tuna.tsinghua.edu.cn/ubuntu/
Suites: noble noble-updates noble-security
Components: main restricted universe multiverse
Signed-By: /usr/share/keyrings/ubuntu-archive-keyring.gpg
```

4. 更新源：

更新源配置并升级系统：

```
sudo apt-get update
sudo apt-get upgrade
```

通过以上步骤，你可以成功将Ubuntu 24.04的源更换为清华源。

2K0300板卡联网

下载终端软件：

<https://mobaxterm.mobatek.net/download-home-edition.html>

1、手动连接网络

2K0300需要先插入天线才能更好和更稳定的连接无线网络。

我们在2K0300的串口终端中输入：

```
wpa_supplicant -B -i wlan0 -c <(wpa_passphrase "账号" "密码")
sleep 3
wpa_supplicant -B -i wlan0 -c <(wpa_passphrase "账号" "密码")
sleep 3
udhcpc -i wlan0
```

// 连接WIFI

```
wpa_supplicant -B -i wlan0 -c <(wpa_passphrase "账号" "密码")
```

// 获取IP

```
udhcpc -i wlan0
```

连接成功后，输入在2K0300的串口终端中输入：`ifconfig`，就可以查看设备的ip地址

```
root@LoongOS:~# ifconfig
lo: flags=73<UP,LOOPBACK,RUNNING> mtu 65536
    inet 127.0.0.1 netmask 255.0.0.0
    inet6 ::1 prefixlen 128 scopeid 0x10<host>
    loop txqueuelen 1000 (Local Loopback)
    RX packets 2418 bytes 183956 (179.6 KiB)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 2418 bytes 183956 (179.6 KiB)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

wlan0: flags=4163<UP,BROADCAST,RUNNING,MULTICAST> mtu 1500
    inet 192.168.2.124 netmask 255.255.255.0 broadcast 192.168.2.255
    inet6 fe80::5e8a:aeff:fe55:72c2 prefixlen 64 scopeid 0x20<link>
    ether 5c:8a:ae:55:72:c2 txqueuelen 1000 (Ethernet)
    RX packets 55970 bytes 59199741 (56.4 MiB)
    RX errors 0 dropped 494 overruns 0 frame 0
    TX packets 4567 bytes 269259 (262.9 KiB)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0
```

通过上面我们可以看到IP地址为192.168.2.124

2、开机自动连接网络

在2K0300的串口终端中输入：`vi /etc/rc.local` 命令

可以看到这里面有如下的代码。

```
#!/bin/bash -e
```

```
do_start()
```

```
{
```

```
#systemctl disable dhcpcd
```

```
#systemctl stop dhcpcd
```

```
insmod /usr/lib/modules/4.19.190+/aic8800_bsp.ko
```

```
insmod /usr/lib/modules/4.19.190+/aic8800_fdrv.ko
```

```
sleep 3
```

```
hostapd -B /etc/hostapd.conf -f /var/log/hstap.log &
```

```
ifconfig wlan0 192.168.1.3
```

```
udhcpd -f /etc/udhcpd.conf &
```

```
}
```

```
do_stop()
```

```
{
```

```
rmmod aic8800_fdrv.ko
```

```
rmmod aic8800_bsp.ko
```

```
}
```

```
case "$1" in
```

```
start)
```

```
do_start
```

```
- /etc/rc.local 1/30 3%
```

```
#!/bin/bash -e
```

```
do_start()
```

```
{
```

```
#systemctl disable dhcpcd
```

```
#systemctl stop dhcpcd
```

```
insmod /usr/lib/modules/4.19.190+/aic8800_bsp.ko
```

```
insmod /usr/lib/modules/4.19.190+/aic8800_fdrv.ko
```

```
sleep 3
```

```
hostapd -B /etc/hostapd.conf -f /var/log/hstap.log &
```

```
ifconfig wlan0 192.168.1.3
```

```
udhcpd -f /etc/udhcpd.conf &
```

```
}
```

```
do_stop()
```

```
{
```

```
rmmod aic8800_fdrv.ko
```

```
rmmod aic8800_bsp.ko
```

```
}
```

```
case "$1" in
```

```
start)
```

```
do_start
```

```
- /etc/rc.local 1/30 3%
```

1

删除

```

#!/bin/bash -e

do_start()
{
#systemctl disable dhcpcd
#systemctl stop dhcpcd
insmod /usr/lib/modules/4.19.190+/aic8800_bsp.ko
insmod /usr/lib/modules/4.19.190+/aic8800_fdrv.ko
}

do_stop()
{
rmmod aic8800_fdrv.ko
rmmod aic8800_bsp.ko
}

case "$1" in
    start)
        do_start
        ;;
    stop)
        do_stop
        ;;
    *)
        ;;
esac

I /etc/rc.local [Modified] 10/25 40%

```

然后添加如下代码：

```

wpa_supplicant -B -i wlan0 -c <(wpa_passphrase "user" "password") &
sleep 3
udhcpc -i wlan0 &
sleep 3

```

```

wpa_supplicant -B -i wlan0 -c <(wpa_passphrase "账号" "密码") &
udhcpc -i wlan0 &

```

这两条命令后面一定需要添加&，否则可能一直卡在开机中。

```
#!/bin/bash -e

do_start()
{
#systemctl disable dhcpcd
#systemctl stop dhcpcd
insmod /usr/lib/modules/4.19.190+/aic8800_bsp.ko
insmod /usr/lib/modules/4.19.190+/aic8800_fdrv.ko

wpa_supplicant -B -i wlan0 -c <(wpa_passphrase "账号" "密码") &
sleep 3
udhcpc -i wlan0 &
sleep 3
}
do_stop()
{
rmmod aic8800_fdrv.ko
rmmod aic8800_bsp.ko
}
case "$1" in
    start)
        do_start
    I /etc/rc.local [Modified] 11/30 36%
```

使用ssh传输文件

1、安装ssh

```
sudo apt-get install openssh-client openssh-server
sudo apt-get install net-tools
```

输入ssh

```
xiaom@ubuntu24:~/桌面$ ssh
usage: ssh [-46AaCfGgKkMMNnqsTtVvXxYy] [-B bind_interface] [-b bind_address]
          [-c cipher_spec] [-D [bind_address:]port] [-E log_file]
          [-e escape_char] [-F configfile] [-I pkcs11] [-i identity_file]
          [-J destination] [-L address] [-l login_name] [-m mac_spec]
          [-O ctl_cmd] [-o option] [-P tag] [-p port] [-R address]
          [-S ctl_path] [-W host:port] [-w local_tun[:remote_tun]]
          destination [command [argument ...]]
          ssh [-Q query_option]
```

2、使用ssh

新建一个SEEKFREE_APP文件

```
xiaom@ubuntu24:~/桌面$ vi SEEKFREE_APP
```

随便输入个内容，我这里输入125

使用cat SEEKFREE_APP指令可以查看该文件的内容。

```
root@LoongOS:~# cat SEEKFREE_APP
125
```

安装教程交叉编译器和OpenCV库

安装之前需要先安装open-vm-tools，安装后可以直接通过WINDOWS和UBUNTU之间，复制粘贴文件。

```
sudo apt-get install open-vm-tools
sudo apt-get install open-vm-tools-desktop
sudo apt-get install git
```

拉取开源库

```
git clone https://gitee.com/seekfree/LS2K0300_Library.git
```

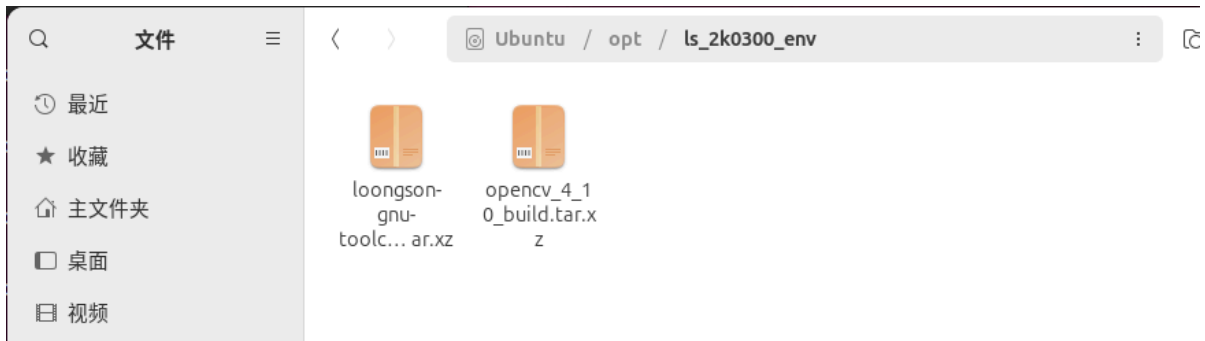
主文件夹 / LS2K0300_Library				大小	修改时间	
名称						
【封装】开源封装				2个项目	今天 14:08	☆
【软件】交叉编译工具 上位机等				3个项目	今天 15:38	☆
【原理图】原理图 丝印图 尺寸图 位号图				5个项目	今天 14:08	☆
Example				4个项目	昨天 18:49	☆
ls2k0300_linux_4.19				34个项目	今天 15:26	☆
Seekfree_LS2K0300_Opensource_Library				3个项目	今天 15:43	☆
LICENSE				35.1 kB	昨天 17:49	☆
README.en.md				969 字节	昨天 17:49	☆
README.md				1.1 kB	昨天 17:49	☆

后面需要使用的opencv_4_10_build.tar.xz和loongson-gnu-toolchain-8.3-x86_64-loongarch64-linux-gnu-rc1.6.tar.xz都在<【软件】交叉编译工具 上位机等>这个文件夹里面

1、ubuntu安装2k0300交叉编译器和OpenCV库

```
cd /opt
sudo mkdir ls_2k0300_env
输入密码
sudo cp opencv_4_10_build.tar.xz /opt/ls_2k0300_env/
输入密码
sudo cp loongson-gnu-toolchain-8.3-x86_64-loongarch64-linux-gnu-rc1.6.tar.xz
/opt/ls_2k0300_env/
输入密码
cd /opt/ls_2k0300_env
输入ls -l:
```

```
xiaom@ubuntu24:/opt/ls_2k0300_env$ ls -l
总计 45172
-rw-r--r-- 1 root root 36028144 3月 5 18:08 loongson-gnu-toolchain-8.3-x86_64-loongarch64-linux-gnu-rc1.6.tar.xz
-rw-rw-r-- 1 xiaom xiaom 10223624 3月 5 18:05 opencv_4_10_build.tar.xz
```



```
sudo tar -xvf opencv_4_10_build.tar.xz
```

```
xiaom@ubuntu24:/opt/ls_2k0300_env$ sudo tar -xvf opencv_4_10_build.tar.xz
opencv_4_10_build/
opencv_4_10_build/share/
opencv_4_10_build/share/licenses/
opencv_4_10_build/share/licenses/opencv4/
opencv_4_10_build/share/licenses/opencv4/flatbuffers-LICENSE.txt
opencv_4_10_build/share/licenses/opencv4/opencvcl-headers-LICENSE.txt
opencv_4_10_build/share/licenses/opencv4/ade-LICENSE
opencv_4_10_build/share/licenses/opencv4/zlib-LICENSE
opencv_4_10_build/share/licenses/opencv4/libjpeg-turbo-README.md
opencv_4_10_build/share/licenses/opencv4/libjpeg-turbo-LICENSE.md
opencv_4_10_build/share/licenses/opencv4/libjpeg-turbo-README.i18n
opencv_4_10_build/share/licenses/opencv4/libtiff-COPYRIGHT
opencv_4_10_build/share/licenses/opencv4/libopenjp2-README.md
opencv_4_10_build/share/licenses/opencv4/libopenjp2-LICENSE
opencv_4_10_build/share/licenses/opencv4/libpng-LICENSE
opencv_4_10_build/share/licenses/opencv4/libpng-README
opencv_4_10_build/share/licenses/opencv4/protobuf-LICENSE
opencv_4_10_build/share/licenses/opencv4/protobuf-README.md
```

```
sudo tar -xvf loongson-gnu-toolchain-8.3-x86_64-loongarch64-linux-gnu-rc1.6.tar.xz
```

```
xiaom@ubuntu24:/opt/ls_2k0300_env$ sudo tar -xvf loongson-gnu-toolchain-8.3-x86_64-loongarch64-linux-gnu-rc1.6.tar.xz
./
loongson-gnu-toolchain-8.3-x86_64-loongarch64-linux-gnu-rc1.6/bin/
loongson-gnu-toolchain-8.3-x86_64-loongarch64-linux-gnu-rc1.6/bin/loongarch64-linux-gnu-addr2line
loongson-gnu-toolchain-8.3-x86_64-loongarch64-linux-gnu-rc1.6/bin/loongarch64-linux-gnu-ar
loongson-gnu-toolchain-8.3-x86_64-loongarch64-linux-gnu-rc1.6/bin/loongarch64-linux-gnu-as
loongson-gnu-toolchain-8.3-x86_64-loongarch64-linux-gnu-rc1.6/bin/loongarch64-linux-gnu-c++
loongson-gnu-toolchain-8.3-x86_64-loongarch64-linux-gnu-rc1.6/bin/loongarch64-linux-gnu-c++filt
loongson-gnu-toolchain-8.3-x86_64-loongarch64-linux-gnu-rc1.6/bin/loongarch64-linux-gnu-cpp
loongson-gnu-toolchain-8.3-x86_64-loongarch64-linux-gnu-rc1.6/bin/loongarch64-linux-gnu-elfedit
loongson-gnu-toolchain-8.3-x86_64-loongarch64-linux-gnu-rc1.6/bin/loongarch64-linux-gnu-g++
loongson-gnu-toolchain-8.3-x86_64-loongarch64-linux-gnu-rc1.6/bin/loongarch64-linux-gnu-gcc
loongson-gnu-toolchain-8.3-x86_64-loongarch64-linux-gnu-rc1.6/bin/loongarch64-linux-gnu-gcc-8.3.0
```

解压完成后，我们输入命令查看该目录下的文件。可以看到有两个压缩文件和两个文件夹。

```
xiaom@ubuntu24:/opt/ls_2k0300_env$ ls -l
总计 45180
drwxr-xr-x 7 root root 4096 3月 5 18:12 loongson-gnu-toolchain-8.3-x86_64-loongarch64-linux-gnu-rc1.6
-rw-r--r-- 1 root root 36028144 3月 5 18:08 loongson-gnu-toolchain-8.3-x86_64-loongarch64-linux-gnu-rc1.6.tar.xz
drwxrwxr-x 6 xiaom xiaom 4096 1月 6 17:58 opencv_4_10_build
-rw-rw-r-- 1 xiaom xiaom 10223624 3月 5 18:05 opencv_4_10_build.tar.xz
```



2、2K0300安装OpenCV库

设备联网后，输入ifconfig，就可以查看设备的ip地址

```
root@LoongOS:~# ifconfig
lo: flags=73<UP,LOOPBACK,RUNNING> mtu 65536
    inet 127.0.0.1 netmask 255.0.0.0
    inet6 ::1 prefixlen 128 scopeid 0x10<host>
    loop txqueuelen 1000 (Local Loopback)
    RX packets 2418 bytes 183956 (179.6 KiB)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 2418 bytes 183956 (179.6 KiB)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

wlan0: flags=4163<UP,BROADCAST,RUNNING,MULTICAST> mtu 1500
    inet 192.168.2.124 netmask 255.255.255.0 broadcast 192.168.2.255
    inet6 fe80::5e8a:aeff:fe55:72c2 prefixlen 64 scopeid 0x20<link>
    ether 5c:8a:ae:55:72:c2 txqueuelen 1000 (Ethernet)
    RX packets 55970 bytes 59199741 (56.4 MiB)
    RX errors 0 dropped 494 overruns 0 frame 0
    TX packets 4567 bytes 269259 (262.9 KiB)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0
```

我们这个设备的IP地址为192.168.2.124。每一个设备的IP地址都不一样，都需要自己去获取一下。

```
scp -Or ./opencv_4_10_build root@192.168.2.124:/home/root/
```

```
xiaom@ubuntu24:/opt/ls_2k0300_env$ scp -Or ./opencv_4_10_build root@192.168.2.124:/home/root/
opencv_model_diagnostics          100% 43KB 407.6KB/s 00:00
opencv_visualisation              100% 43KB 456.0KB/s 00:00
opencv_annotation                 100% 42KB 419.0KB/s 00:00
opencv_interactive-calibration    100% 165KB 834.1KB/s 00:00
setup_vars_opencv4.sh            100% 671 165.9KB/s 00:00
opencv_version                    100% 43KB 397.9KB/s 00:00
libopencv_stitching.so            0% 0 0.0KB/s --:-- ETA^
libopencv_stitching.so            100% 754KB 909.4KB/s 00:00
libopencv_dnn.so.4.10.0           79% 6336KB 1.1MB/s 00:01 ETA
```

等待scp传输完成。

```
flatbuffers-LICENSE.txt          100% 11KB 137.0KB/s 00:00
zlib-LICENSE                     100% 1002 67.6KB/s 00:00
protobuf-LICENSE                 100% 1732 27.9KB/s 00:00
opencv-headers-LICENSE.txt       100% 1309 336.6KB/s 00:00
libpng-README                    100% 9454 134.8KB/s 00:00
libopenjp2-README.md             100% 5402 317.9KB/s 00:00
SoftFloat-COPYING.txt            100% 1732 28.0KB/s 00:00
mscr-chi_table_LICENSE.txt        100% 1559 12.3KB/s 00:00
libjpeg-turbo-README.ijg         100% 12KB 195.7KB/s 00:00
ade-LICENSE                      100% 11KB 124.9KB/s 00:00
libjpeg-turbo-LICENSE.md         100% 5620 269.0KB/s 00:00
xiaom@ubuntu24:/opt/ls_2k0300_env$
```

3、在2K0300中添加OpenCV索引

打开etc目录

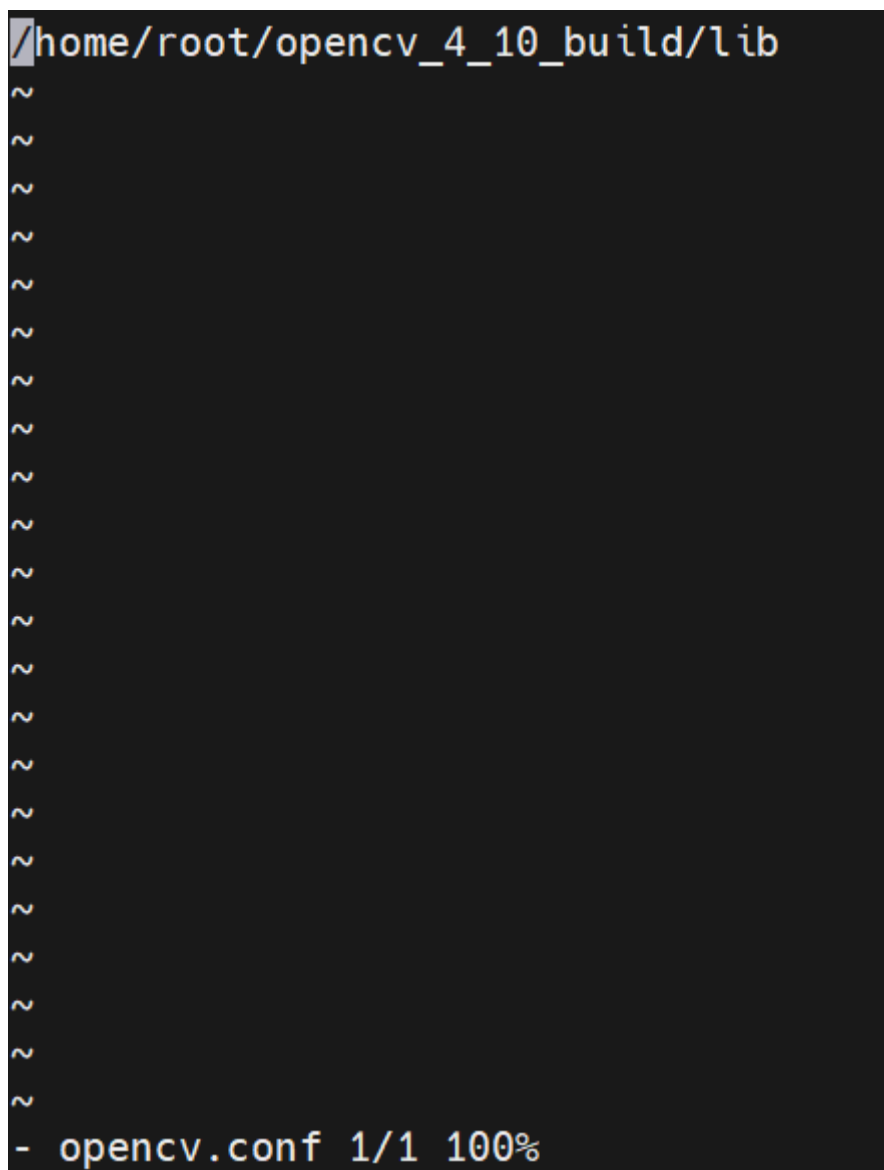
```
cd /etc/
```

添加ld.so.conf.d目录，并打开它

```
mkdir ld.so.conf.d  
cd ld.so.conf.d/
```

新建并编辑，opencv.conf文件

```
vi opencv.conf
```



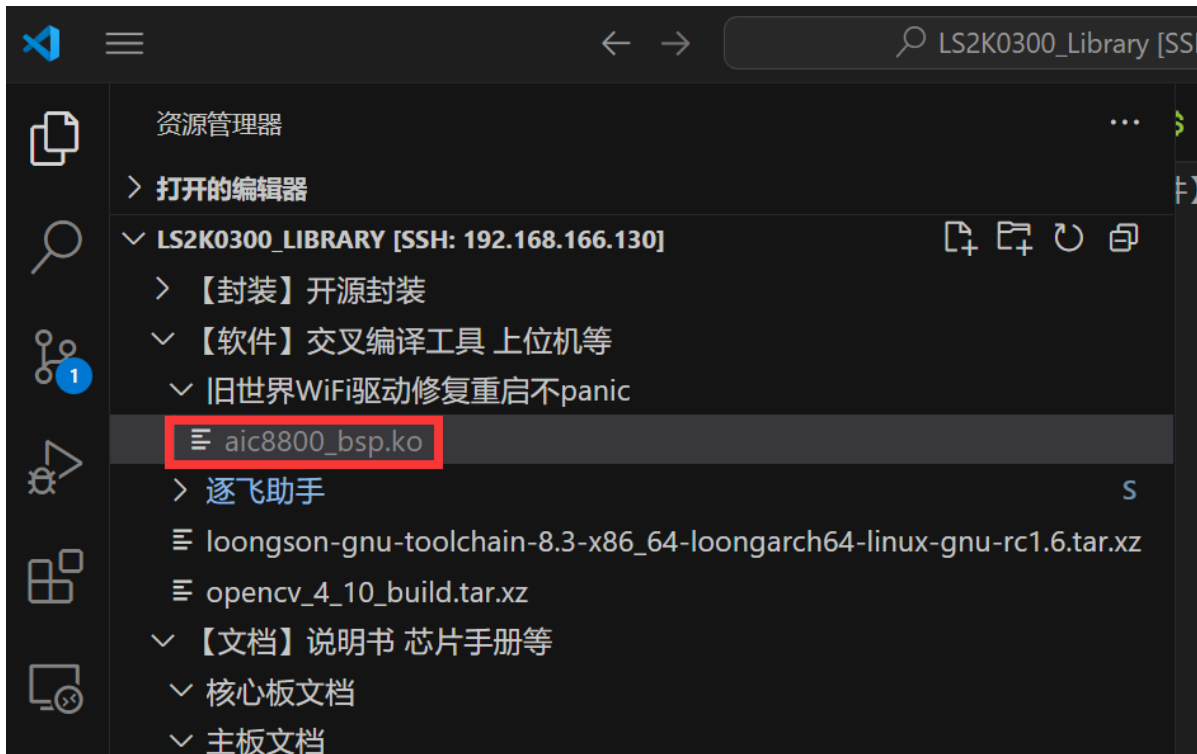
输入

```
/home/root/opencv_4_10_build/lib
```

然后保存退出

最后输入ldconfig，即可。

关机或重启久久派卡panic解决方案



在我们提供的资料中，有一个aic8800_bsp.ko文件，这个文件是网卡驱动文件。我们需要替换到久久派的板卡上面。

1、使用reboot卡住

我们在久久派板卡中，输入reboot，就会提示：

```
4. COM21 (USB 串行设备 (COM21) x
[51719.832305] t4 00000000048082c0 t5 0000000000000000 t6 0000000000000386 t7 0000000000000007
[51719.840682] t8 00000000000bc67f u0 9000000001470000 s9 900000009c451070 s0 900000009cade408
[51719.849060] s1 900000009d483900 s2 ffffffff8002025cc0 s3 ffffffff8002025430 s4 0000000000000060
[51719.857437] s5 9000000001470000 s6 0000000000000001 s7 900000000112e450 s8 900000000112e440
[51719.865891] ra: ffffffff800201adfc aicbsp_sdio_remove+0x50/0x1a0 [aic8800_bsp]
[51719.873253] ERA: ffffffff800201ae7c aicbsp_sdio_remove+0xd0/0x1a0 [aic8800_bsp]
[51719.880587] CRMD: 900000009cade408 (PLV0 -IE +DA -PG DACF=SUC DACM=SUC -WE)
[51719.887657] PRMD: 00000004 (PPLV0 +PIE -PWE)
[51719.892024] EUEN: 00000001 (+FPE -SXE -ASXE -BTE)
[51719.896828] ECFG: 00000000 (LIE= VS=0)
[51719.900675] ESTAT: 9000000096dc0ff8 [???] (IS=3-11 ECode=28 EsubCode=91)
[51719.907391] PRID: 0014a030 (Loongson-64bit, )
[51719.911842] Modules linked in: aic8800_fdrv(O) aic8800_bsp(O)
[51719.917607] Process systemd-shutdown (pid: 1, threadinfo=00000000ec954ab0, task=0000000040dfd094)
[51719.926415] Stack : ffffffff8002025430 900000009cade400 900000009cade408 900000000acd938
[51719.934449] 0000000000000000 900000009cade468 900000009cade408 9000000008da070
[51719.942481] 900000009cade418 900000009ca63008 90000000012f8b60 900000009cade408
[51719.950513] 900000009c3e4c58 90000000008d88cc 900000009c2ec000 900000009cade468
[51719.958545] 900000009cade408 90000000008d52e8 00000000810000f6 0000000000000001
[51719.966577] 9000000001470000 900000000112e450 0000000000000001 9000000001470000
[51719.974608] 0000000000000001 90000000014da340 900000009c2ec000 0000000000000002
[51719.982640] 900000009cade408 9000000000ace0c0 00000000000004c8 9000000000acb8f0
[51719.990672] 900000009c2ec384 00000000000000b4 900000009c2ec000 9000000000ac2e0c
[51719.998704] 900000009c451010 900000009c2aec10 900000009c2ec000 9000000000ac4174
[51720.006736] ...
[51720.009194] Call Trace:
[51720.011677] [<fffffff800201ae7c>] aicbsp_sdio_remove+0xd0/0x1a0 [aic8800_bsp]
[51720.018774] [<9000000000acd938>] sdio_bus_remove+0x48/0x110
[51720.024365] [<90000000008da070>] device_release_driver_internal+0x1c0/0x250
[51720.031341] [<90000000008d88cc>] bus_remove_device+0x12c/0x140
[51720.037190] [<90000000008d52e8>] device_del+0x148/0x360
[51720.042427] [<9000000000ace0c0>] sdio_remove_func+0x30/0x50
[51720.048011] [<9000000000acb8f0>] mmc_sdio_remove+0x50/0x90
[51720.053512] [<9000000000ac2e0c>] mmc_stop_host+0x19c/0x290
[51720.059012] [<9000000000ac4174>] mmc_remove_host+0x24/0x50
[51720.064508] [<90000000008d6ef4>] device_shutdown+0x1a4/0x2a0
[51720.070186] [<900000000024568c>] kernel_restart+0x1c/0x70
[51720.075598] [<9000000000245aa8>] sys_reboot+0x208/0x250
[51720.080836] [<90000000002116f8>] syscall_common+0x24/0x34
[51720.086244] Code: 02bbfc0e 28c0018c 28c0018d <288d71ac> 0014b98c 298d71ac 28c00317 440046e0 140
404ac
[51720.096021]
[51720.097568] ---[ end trace 4e2566dc5893c65 ]---
[51720.102359] Kernel panic - not syncing: Attempted to kill init! exitcode=0x0000000b
[51720.102359]
[51720.111547] ---[ end Kernel panic - not syncing: Attempted to kill init! exitcode=0x0000000b
[51720.111547] ]---
```

然后，就卡住。

这里，我们需要按一下板卡上面的复位按键，重启一下。

2、替换aic8800_bsp.ko

在前面的章节，我们讲解了如何使用ssh传输。

2.1、使用ssh传输



我们在<旧世界WiFi驱动修复重启不panic>文件夹里面，打开终端，然后输入

```
scp -O aic8800_bsp.ko root@192.168.2.57:/lib/modules/4.19.190+
```

即可替换，该驱动文件。

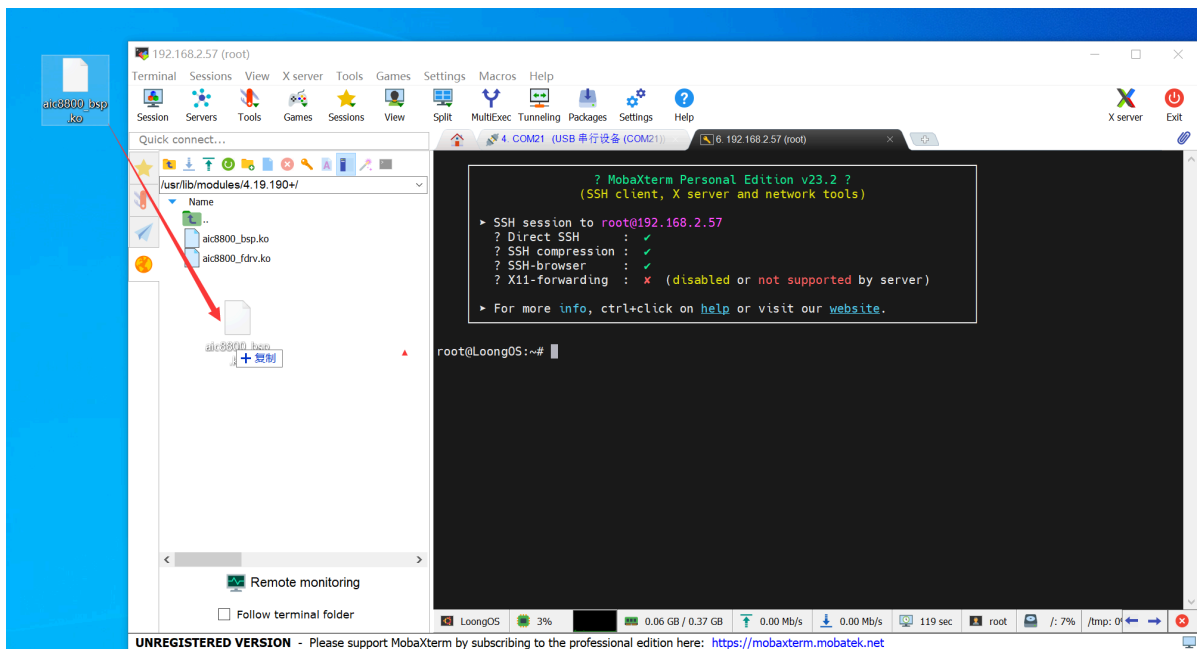
替换完成后，一定要输入sync同步一下，防止没有替换完成。

sync



2.2、使用mobaxterm终端工具传输

这里我们可以通过mobaxterm软件，打开ssh连接。将文件直接拖入久谦派板卡中，即可替换该驱动文件。



编译开源库

1、拉取开源库

在前面，我们已经拉取开源库了。

```
git clone https://gitee.com/seekfree/LS2K0300_Library.git
```

主文件夹 / LS2K0300_Library						
名称		大小	修改时间			
【封装】开源封装		2个项目	今天 14:08	☆		
【软件】交叉编译工具 上位机等		3个项目	今天 15:38	☆		
【原理图】原理图 丝印图 尺寸图 位号图		5个项目	今天 14:08	☆		
Example		4个项目	昨天 18:49	☆		
ls2k0300_linux_4.19		34个项目	今天 15:26	☆		
Seekfree_LS2K0300_Opensource_Library		3个项目	今天 15:43	☆		
LICENSE		35.1 kB	昨天 17:49	☆		
README.en.md		969 字节	昨天 17:49	☆		
README.md		1.1 kB	昨天 17:49	☆		

2、编译开源库

打开LS2K0300_Library/Seekfree_LS2K0300_Opensource_Library/project/user目录

主文件夹 / LS2K0300_Library / Seekfree_LS2K0300_Opensource_Library / project / user						
名称						
build.sh						
CMakeLists.txt						
cross.cmake						
main.cpp						

可以看到里面有四个文件:

build.sh是编译脚本，直接运行这个就可以编译应用程序。

CMakeLists.txt 是使用 CMake 构建系统时的核心配置文件，用于描述项目的构建规则，CMake 会依据其中信息生成对应平台的构建文件，像 Makefile、Visual Studio 解决方案等，进而完成项目的编译和构建

cross.cmake 通常是一个用于交叉编译的 CMake 工具链文件。

main.cpp是用户编写程序的地方。

安装必要的软件，提前装好。

```
sudo apt-get install open-vim-tools
sudo apt-get install open-vim-tools-desktop
sudo apt-get install git
sudo apt-get install cmake
```

编译前准备，安装一系列的软件。

安装完成后，编译代码。

编译代码的方式一

导出交叉编译器

```
export PATH=/opt/ls_2k0300_env/loongson-gnu-toolchain-8.3-x86_64-loongarch64-  
linux-gnu-rc1.6/bin:$PATH
```

使用cmake指令生成makefile文件

```
cmake .
```

编译

```
make
```

编译代码的方式二

直接运行./build.sh即可生成

```
./build.sh
```

编译内核

内核的源码是为了提供给那些需要改动引脚或者修改驱动的同学使用的。

在你不需要修改设备树或者驱动的时候，我们并不需要去编译内核。直接使用编译好的内核即可。

安装必要的软件，提前装好。

```
sudo apt-get install open-vim-tools  
sudo apt-get install open-vim-tools-desktop  
sudo apt-get install git  
sudo apt-get install cmake  
sudo apt-get install bison  
sudo apt-get install flex  
sudo apt-get install libssl-dev
```

1、拉取开源库

在前面，我们已经过拉取开源库了。

```
git clone https://gitee.com/seekfree/LS2K0300_Library.git
```

主文件夹 / LS2K0300_Library						
名称	大小	修改时间				
【封装】开源封装	2个项目	今天 14:08	☆			
【软件】交叉编译工具 上位机等	3个项目	今天 15:38	☆			
【原理图】原理图 丝印图 尺寸图 位号图	5个项目	今天 14:08	☆			
Example	4个项目	昨天 18:49	☆			
ls2k0300_linux_4.19	34个项目	今天 15:26	☆			
Seekfree_LS2K0300_Opensource_Library	3个项目	今天 15:43	☆			
LICENSE	35.1 kB	昨天 17:49	☆			
README.en.md	969 字节	昨天 17:49	☆			
README.md	1.1 kB	昨天 17:49	☆			

其中ls2k0300_linux_4.19目录为linux 4.19版本的内核。

进入这个目录，然后运行终端。

2、尝试编译内核

导出交叉编译器

```
export PATH=/opt/ls_2k0300_env/loongson-gnu-toolchain-8.3-x86_64-loongarch64-  
linux-gnu-rc1.6/bin:$PATH
```

编译，其中的-j12代表使用12线程编译。这个多少线程得根据ubuntu的线程数来分配。如果ubuntu中最大支持10线程，那么使用make最多使用-j10来进行编译。

```
make -j12
```

在第一次编译内核的时候，这个过程可能会持续很久的时间，需要耐心等待一下。

```
xiaom@ubuntu24: ~/LS2K0300_Library/ls2k0300_linux_4.19
xiaom@ubuntu24:~/LS2K0300_Library/ls2k0300_linux_4.19$ export PATH=/opt/ls_2k0300_
0_env/loongson-gnu-toolchain-8.3-x86_64-loongarch64-linux-gnu-rc1.6/bin:$PATH
xiaom@ubuntu24:~/LS2K0300_Library/ls2k0300_linux_4.19$ make -j12
  CALL  scripts/checksyscalls.sh
  CHK   include/generated/compile.h
^[[^Amake[4]: 警告：子 make 中强制 -j16：重置 jobserver 模式。
make[5]: 警告：子 make 中强制 -j16：重置 jobserver 模式
```

编译完成后就会出现vmlinuz这个文件，这个文件就是内核文件。

主文件夹 / LS2K0300_Library / ls2k0300_linux_4.19			大小	修改时间
名称				
Kconfig	563 字节	昨天 21:04	☆	
MAINTAINERS	471.9 kB	昨天 21:04	☆	
Makefile	60.7 kB	昨天 21:04	☆	
Module.symvers	611.8 kB	今天 11:30	☆	
modules.builtin	10.5 kB	今天 11:28	☆	
modules.order	3.7 kB	今天 11:30	☆	
README	800 字节	昨天 21:04	☆	
README.md	0 字节	昨天 21:04	☆	
System.map	4.2 MB	今天 11:30	☆	
vmlinux	24.3 MB	今天 11:30	☆	
vmlinux.o	92.5 MB	今天 11:30	☆	
vmlinuz	6.1 MB	今天 11:30	☆	
vmlinuz.efi	6.0 MB	今天 11:30	☆	

龙芯2K0300久久派板卡，内核(vmlinuz)文件存放在/boot文件夹下面。

```
root@LoongOS:boot# pwd
/boot
root@LoongOS:boot# ll
total 10784
-rwxr-xr-x 1 root root 450 Mar 24 10:32 boot.cfg
-rwxr-xr-x 1 root root 6050304 Mar 25 03:19 vmlinuz
-rw-r--r-- 1 root root 4977152 Jan 1 2020 vmlinuz-4.19.190-rt79-yocto-standard
```

在这个文件夹下面有三个文件，第一个boot.cfg为启动配置文件：

```
timeout 3
default 0
showmenu 1

title 'LoongOS (2k300)'
kernel (emmc0,0)/boot/vmlinuz
#initrd (usb0,0)/boot/LoongOS.cpio.gz.cpio.gz
args console=tty console=ttyS0,115200 rdinit=/sbin/init rootdelay=5 root=/dev/mmcblk0p1
```

第二个vmlinuz文件为内核。

第三个文件没有任何作用，可以直接删掉。

3、更新内核

3.1、使用ssh传输

内核的更新可以使用ssh中的scp -O命令，把文件传输到对应的目录下即可。

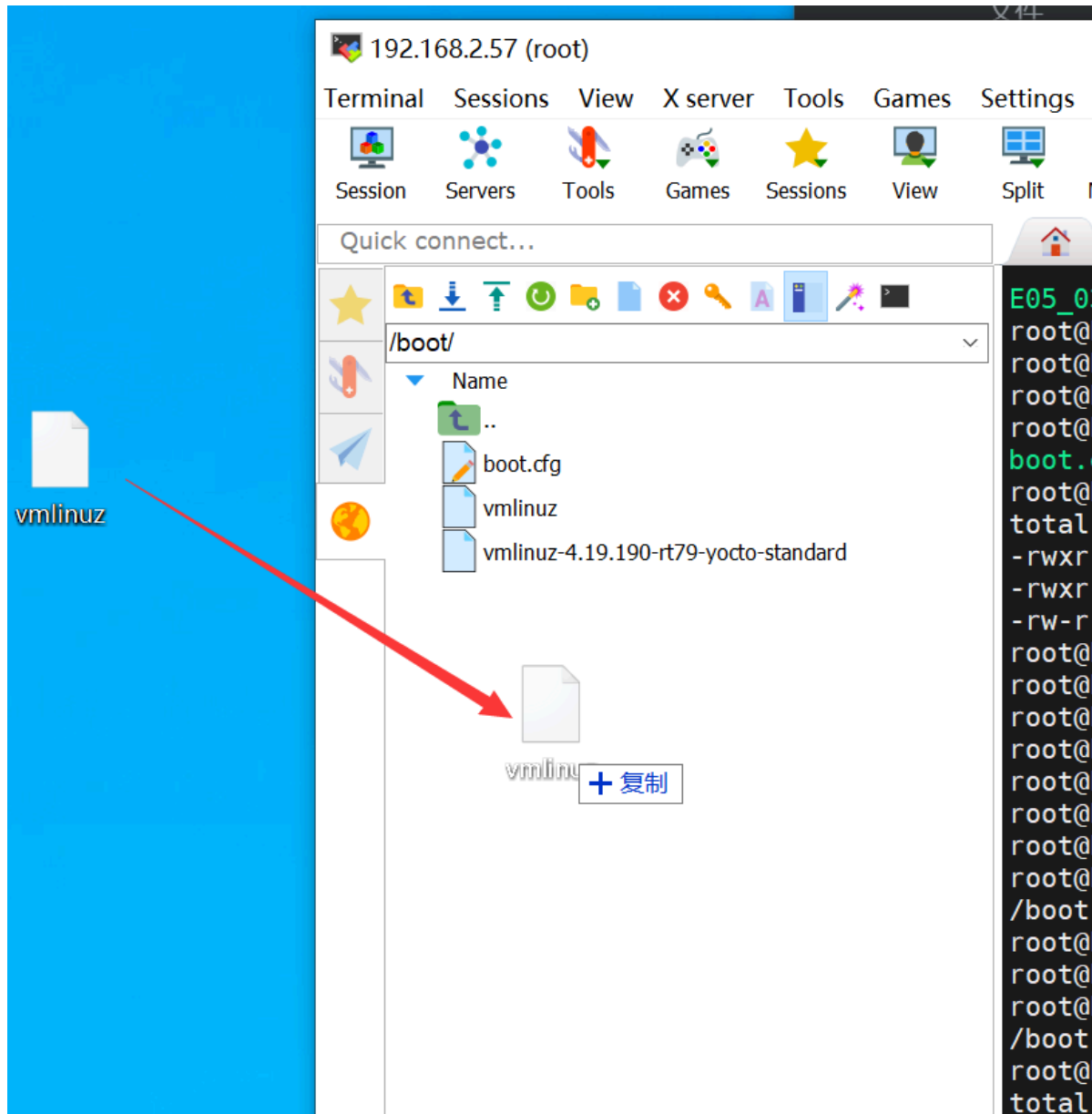
```
scp -O vmlinuz root@192.168.2.57:/boot
```

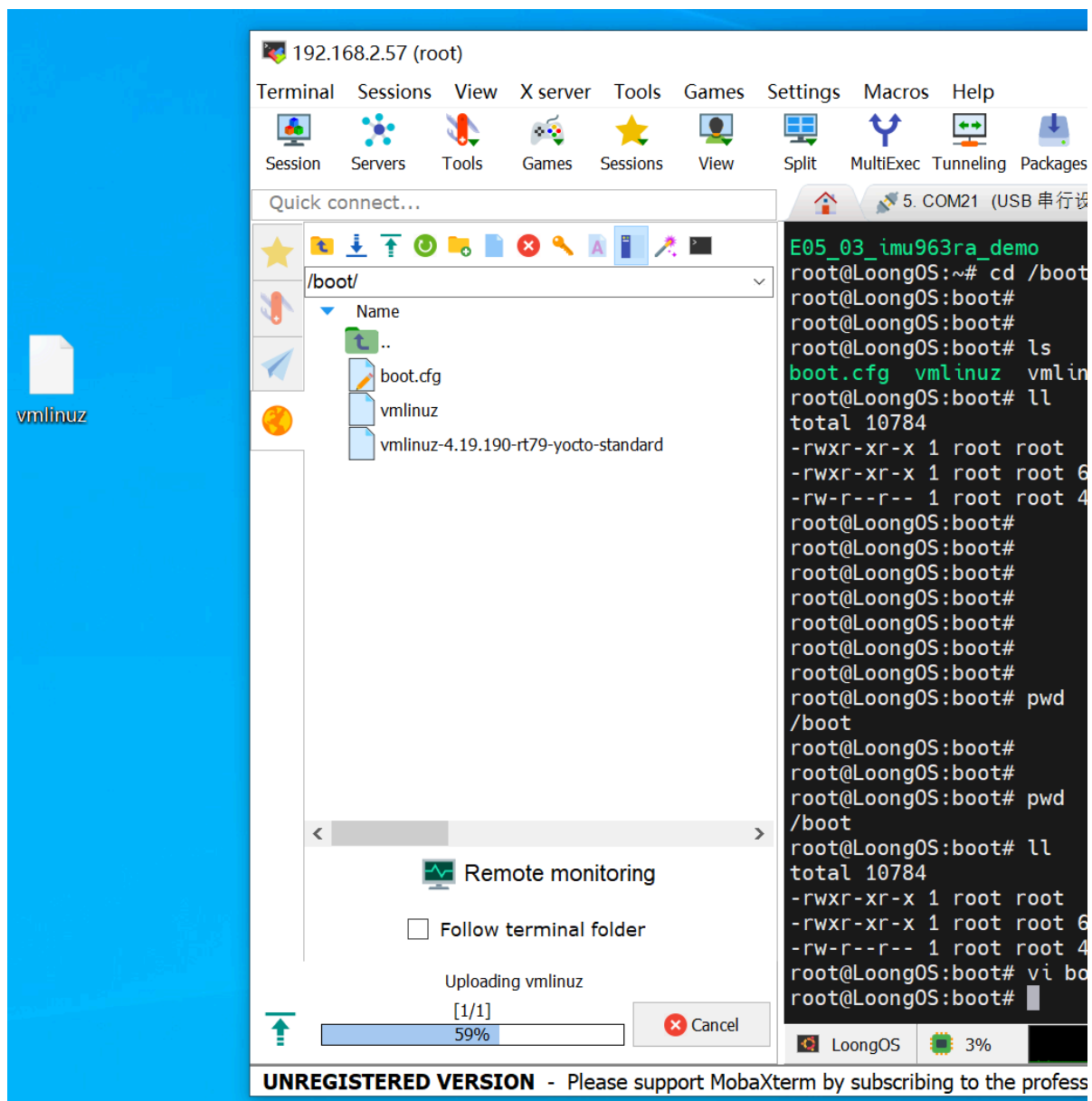
```
xiaom@ubuntu24:~/LS2K0300_Library/ls2k0300_linux_4.19$ ll vmlin*
-rwxrwxr-x 1 xiaom xiaom 24289320 3月 8 11:30 vmlinuz*
-rw-rw-r-- 1 xiaom xiaom 92489272 3月 8 11:30 vmlinuz.o
-rwxrwxr-x 1 xiaom xiaom 6050304 3月 8 11:30 vmlinuz*
-rwxrwxr-x 1 xiaom xiaom 6033920 3月 8 11:30 vmlinuz.efi*
xiaom@ubuntu24:~/LS2K0300_Library/ls2k0300_linux_4.19$ scp -O vmlinuz root@192.168.2.57:/boot
vmlinuz
100% 5909KB 921.3KB/s 00:06
xiaom@ubuntu24:~/LS2K0300_Library/ls2k0300_linux_4.19$
```

传输完成，一定要输入sync进行同步，否则可能会出现内核没有复制完成，导致一直卡PMON中。

3.2、使用mobaxterm终端工具传输

或者可以使用mobaxterm工具，直接将vmlinuz拖入该目录(\boot)下。





等待复制完成，一定要输入sync同步一下，再输入reboot。

```
root@LoongOS:boot# sync
root@LoongOS:boot# reboot
```

否则就可能出现，内核没有复制完成，重启后无法加载内核，启动失败的情况。

安装vscode，使用ssh连接ubuntu

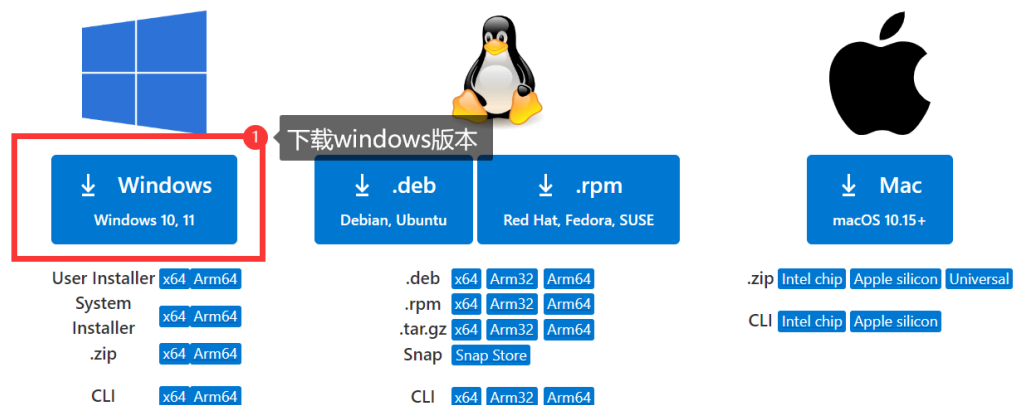
本章节主要讲解的是，在windows里面安装vscode。然后使用windows的vscode通过ssh连接ubuntu，并打开ubuntu中的文件夹。

1、windows安装vscode，使用ssh连接ubuntu

下载链接：<https://code.visualstudio.com/Download>

Download Visual Studio Code

Free and built on open source. Integrated Git, debugging and extensions.



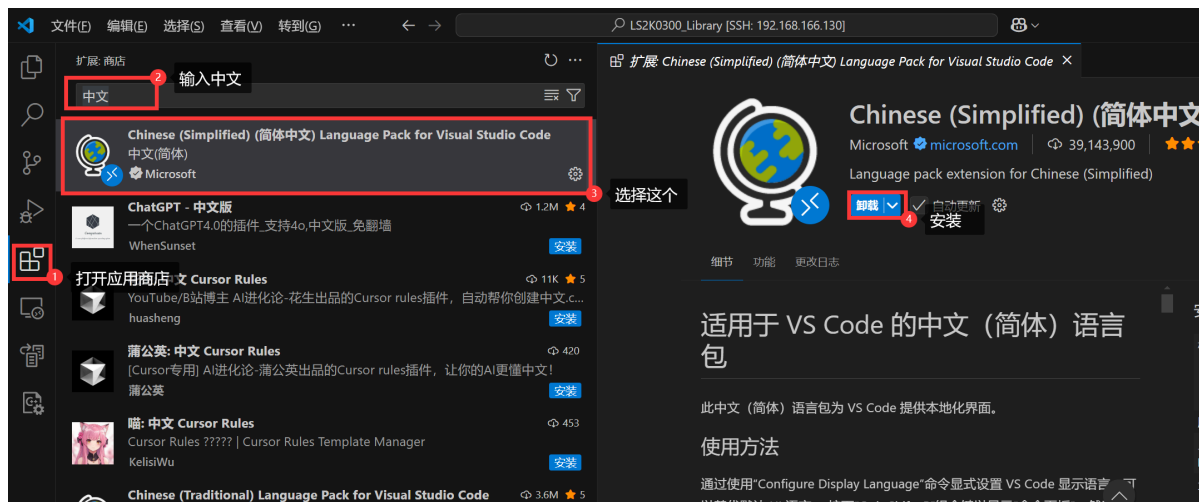
安装，就是一直点下一步，就可以完成安装。

2、windows使用ssh连接ubuntu

ubuntu需要提前安装

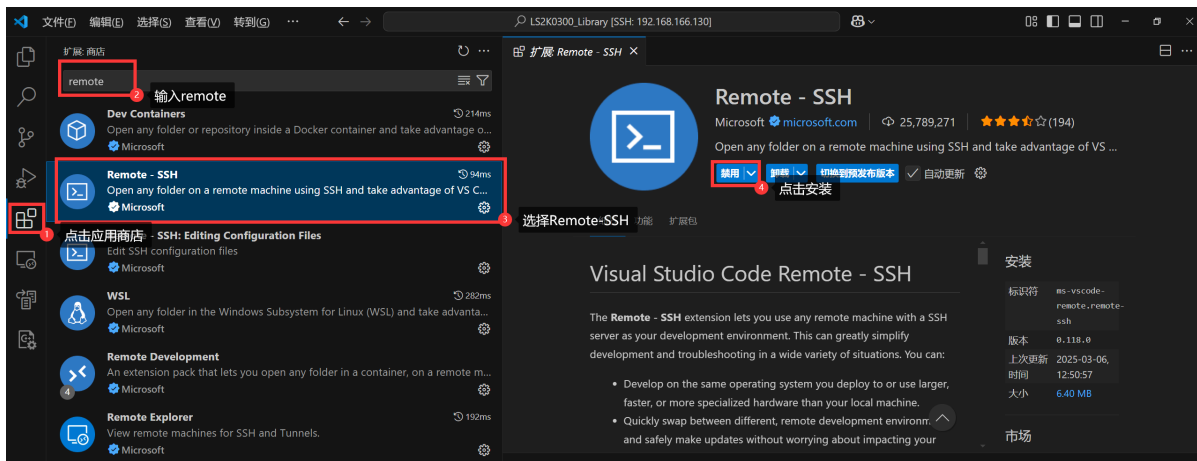
```
sudo apt-get install openssh-client openssh-server
```

2.1、vscode安装中文



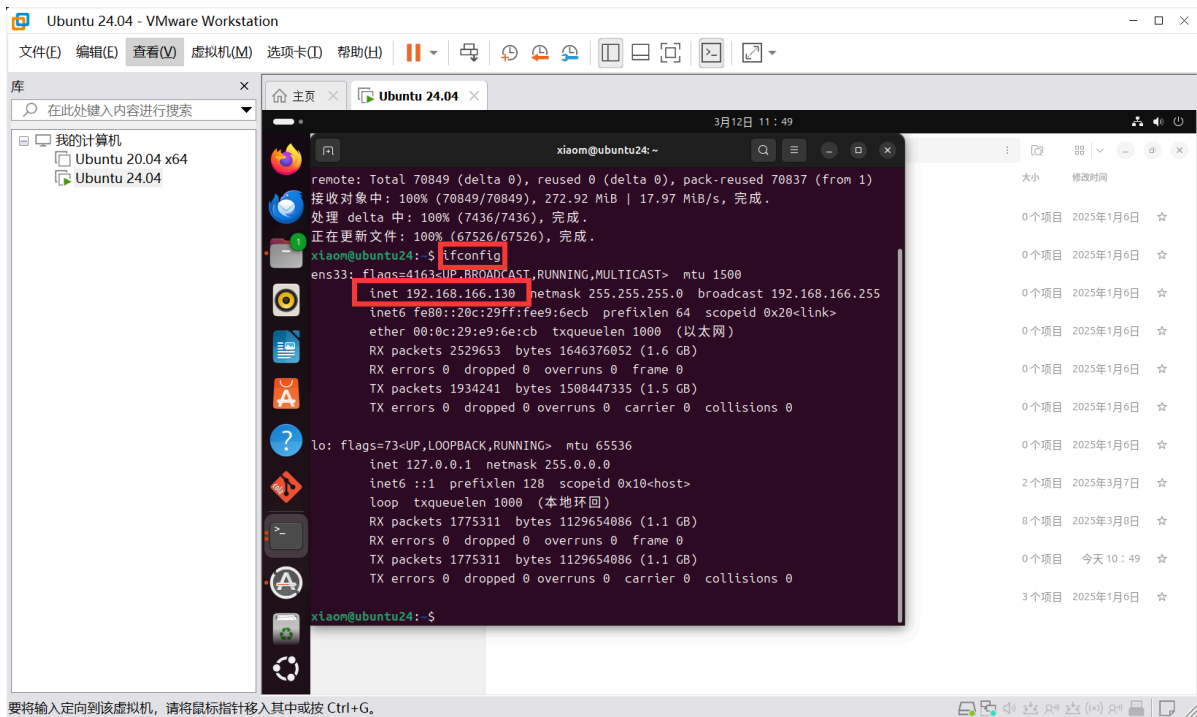
2.2、vscode安装Remote-SSH

打开vscode安装Remote-SSH



2.3、vscode使用ssh连接ubuntu

查看ubuntu的ip地址，使用ifconfig指令。



如果没有ifconfig命令，就需要安装：

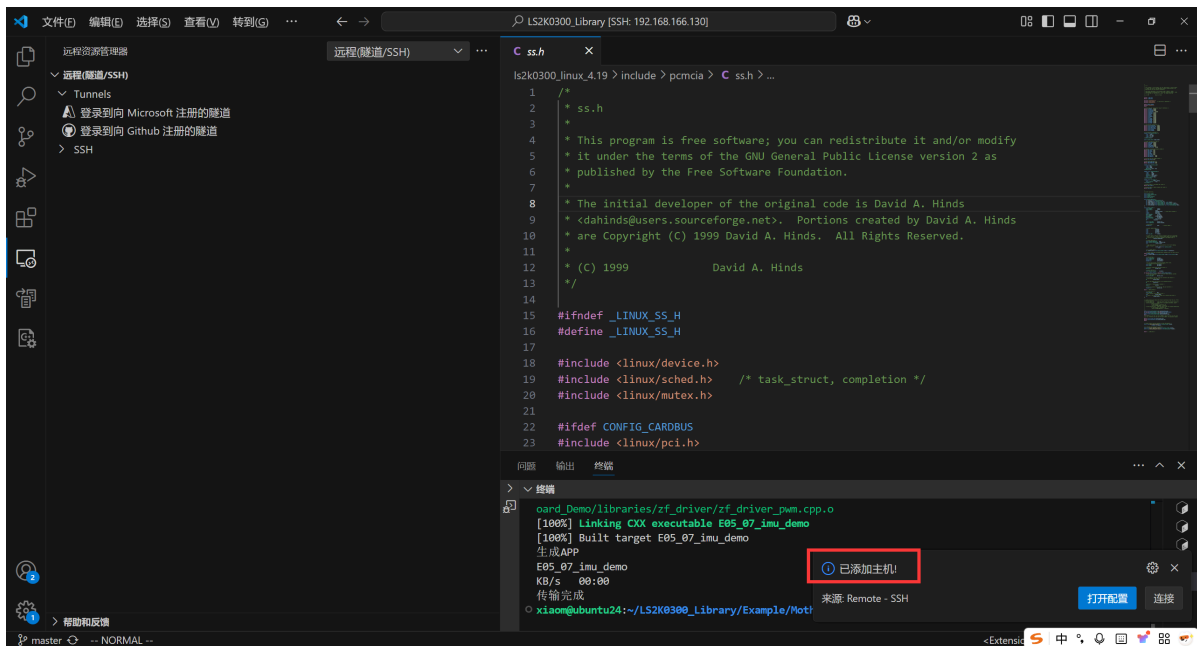
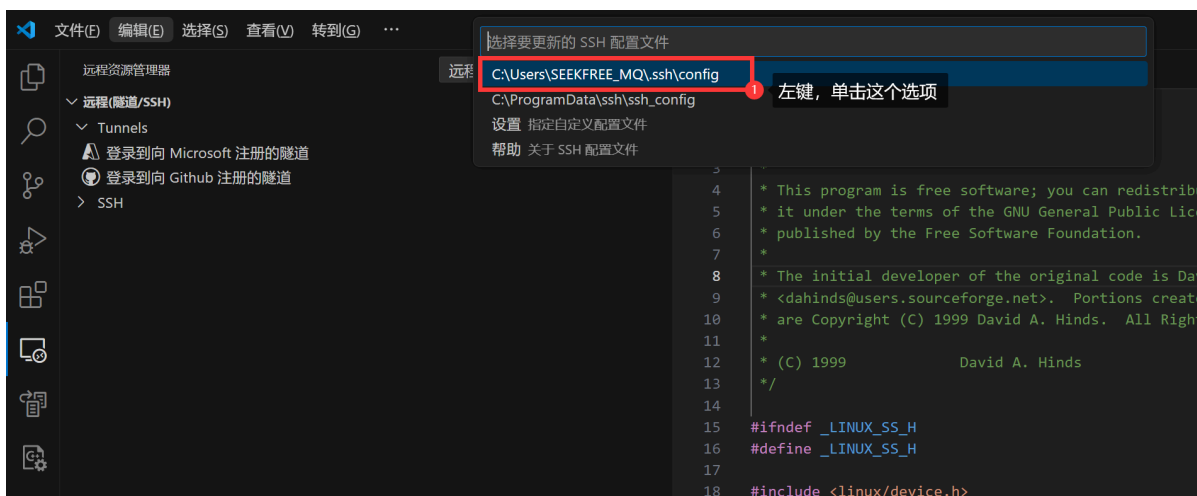
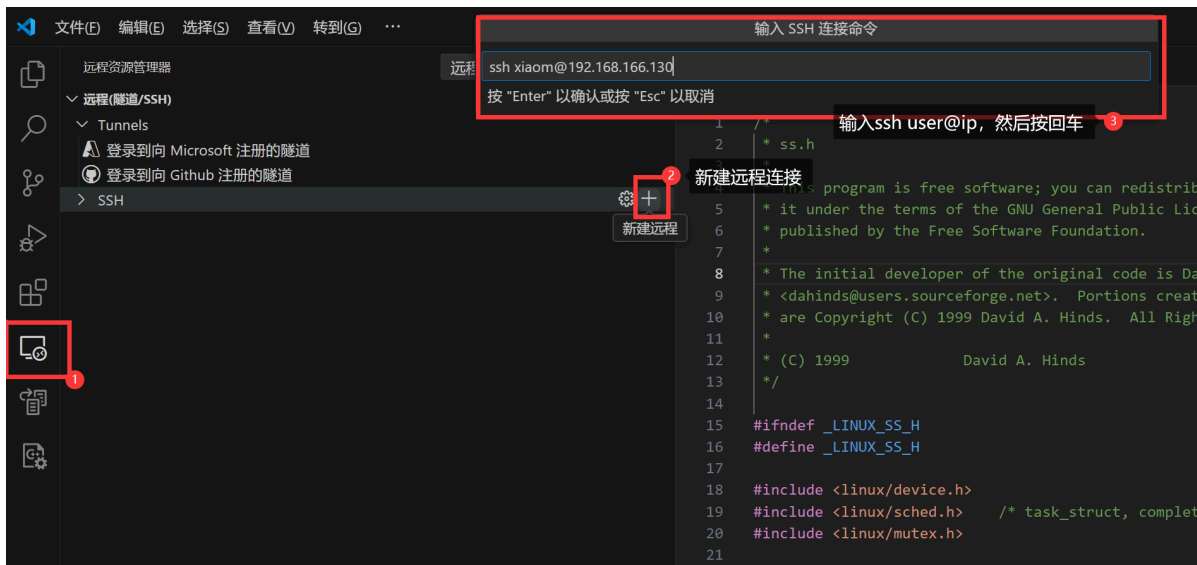
```
sudo apt-get install net-tools
```

我这个ubuntu的用户名为：xiaom。ip地址为：192.168.166.130

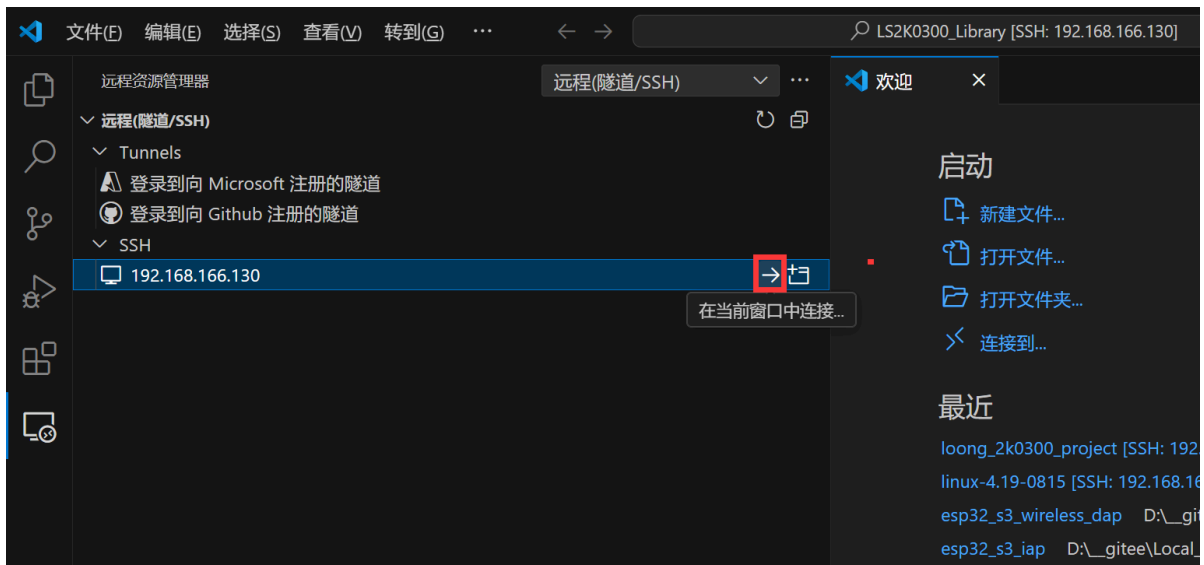
我就应该输入

```
ssh xiaom@192.168.166.130
```

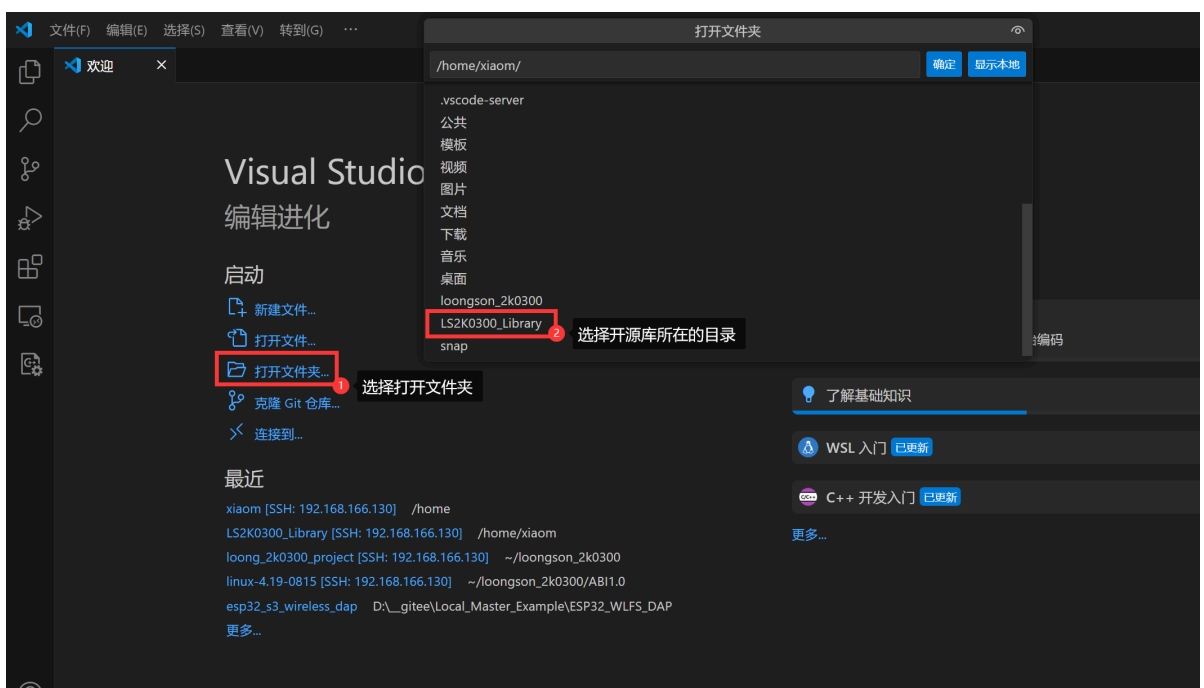
如果是其他用户名和ip地址，按照 ssh user@ip 输入即可。

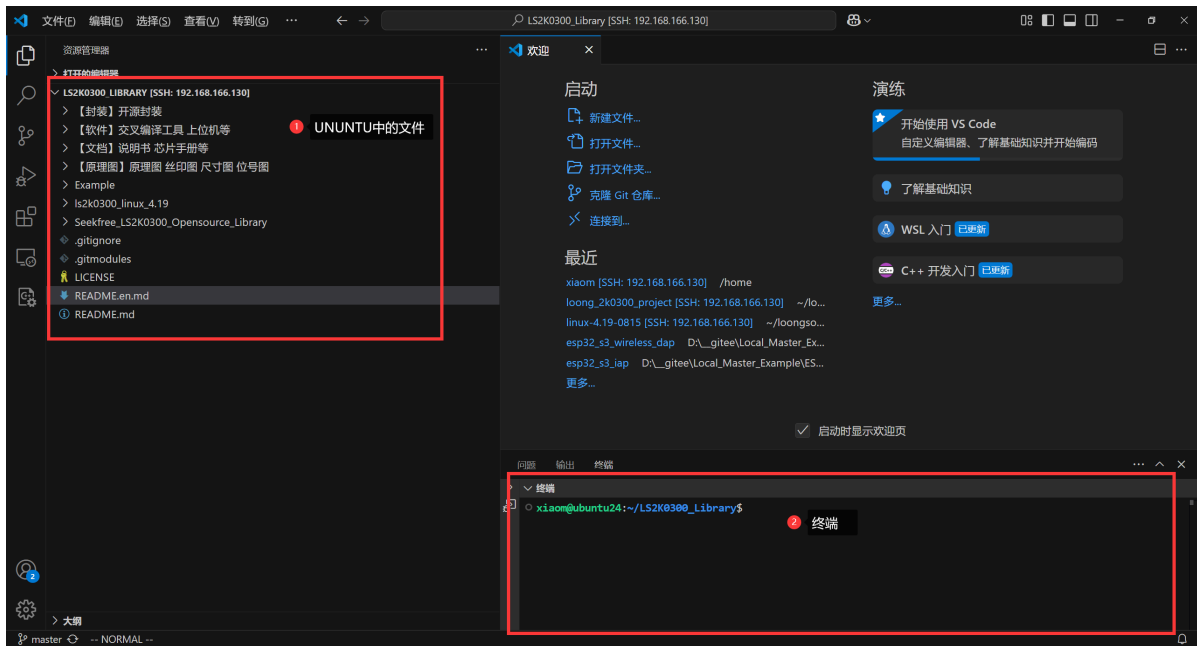
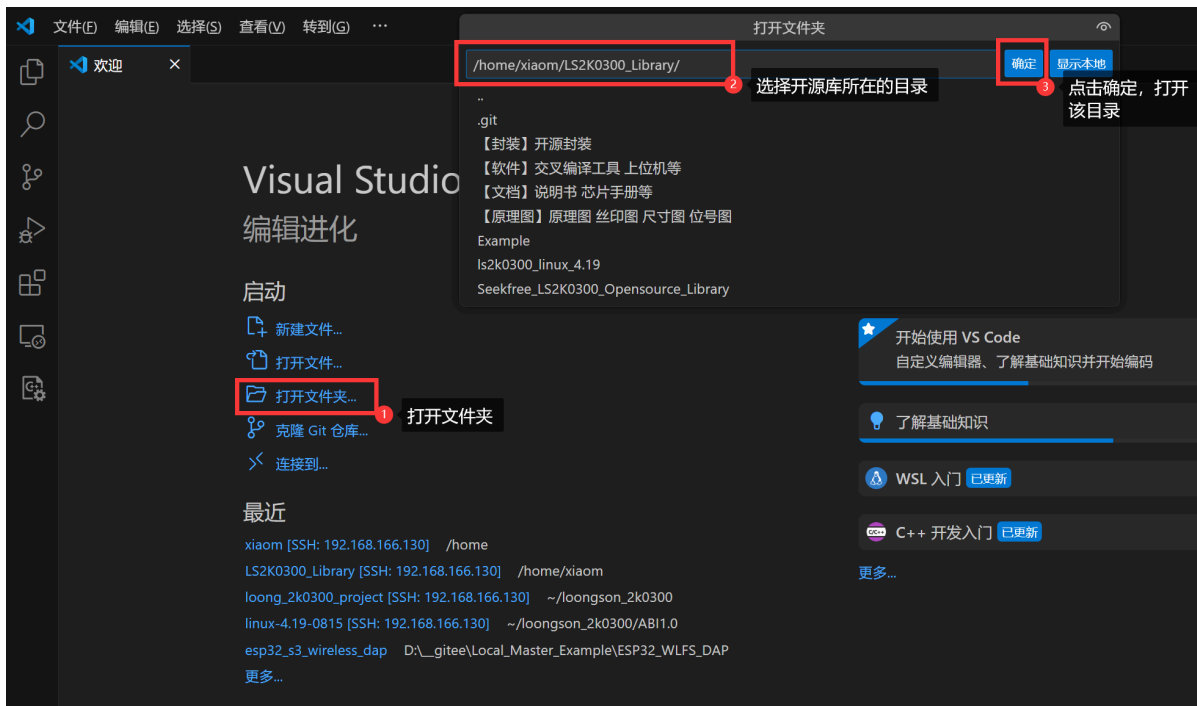


这里提示已添加主机，则添加成功。



3、windows使用vscode打开开源库



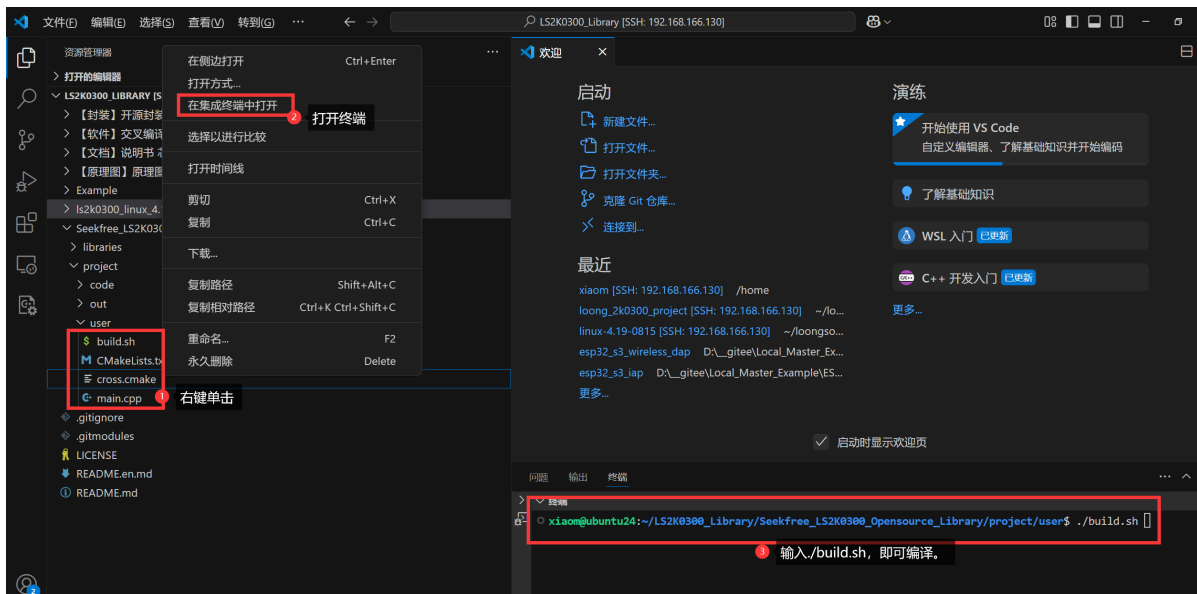


4、windows使用vscode编译开源库和内核

必要的软件，需要提前安装好。

```
sudo apt-get install open-vm-tools
sudo apt-get install open-vm-tools-desktop
sudo apt-get install git
sudo apt-get install cmake
sudo apt-get install bison
sudo apt-get install flex
sudo apt-get install libssl-dev
sudo apt-get install libncurses5-dev
```

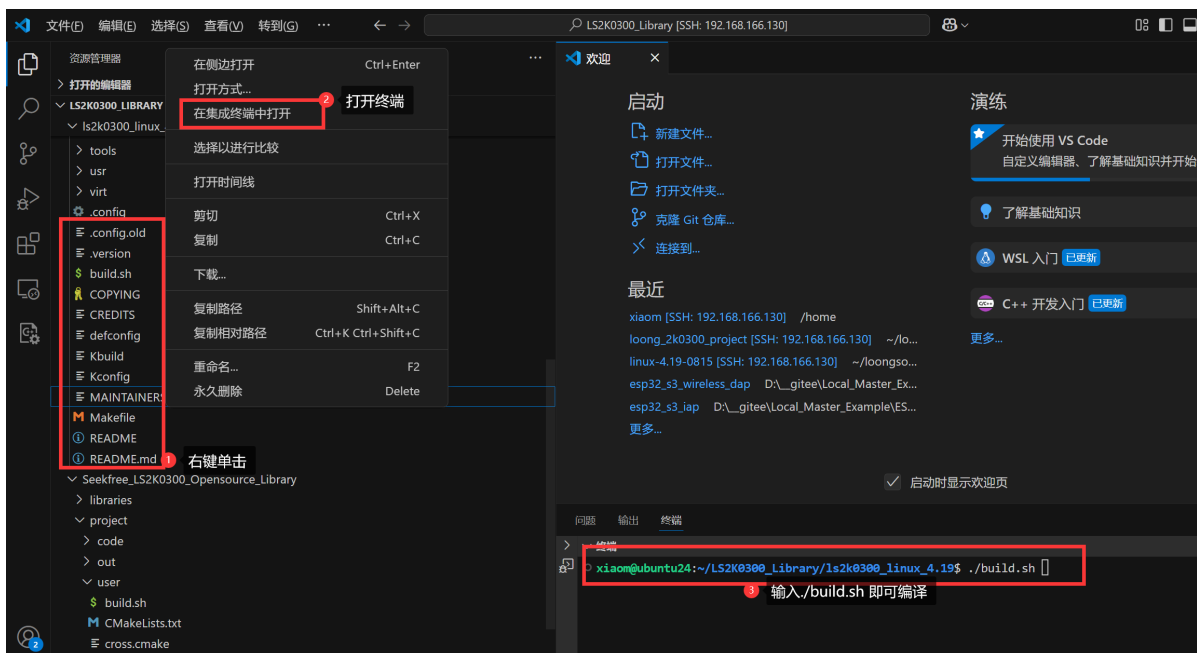
打开user目录，然后运行编译脚本即可编译开源库。



也可以使用下面命令进行编译。

```
export PATH=/opt/ls_2k0300_env/loongson-gnu-toolchain-8.3-x86_64-loongarch64-  
linux-gnu-rc1.6/bin:$PATH  
cmake .  
make -j12
```

打开内核文件夹根目录，然后运行编译脚本即可编译开源库。



也可以使用命令编译内核

```
export PATH=/opt/ls_2k0300_env/loongson-gnu-toolchain-8.3-x86_64-loongarch64-  
linux-gnu-rc1.6/bin:$PATH  
make -j12
```

设备树讲解

linux引入设备树的目的是用统一、灵活且便于维护的方式来描述硬件设备信息，让内核能够更好地适配不同的硬件平台和配置变动。如此一来，诸如引脚定义、引脚复用以及引脚配置等功能都能在设备树里统一设置。要是需要对引脚进行修改，仅调整设备树就可以了。

当前，我们所添加的驱动是直接编译到内核里的。在 IMU、屏幕等设备启动时，会对它们进行初始化。但如果在设备启动后再插入，这些设备就无法被初始化，此时就必须重启久久派板卡。

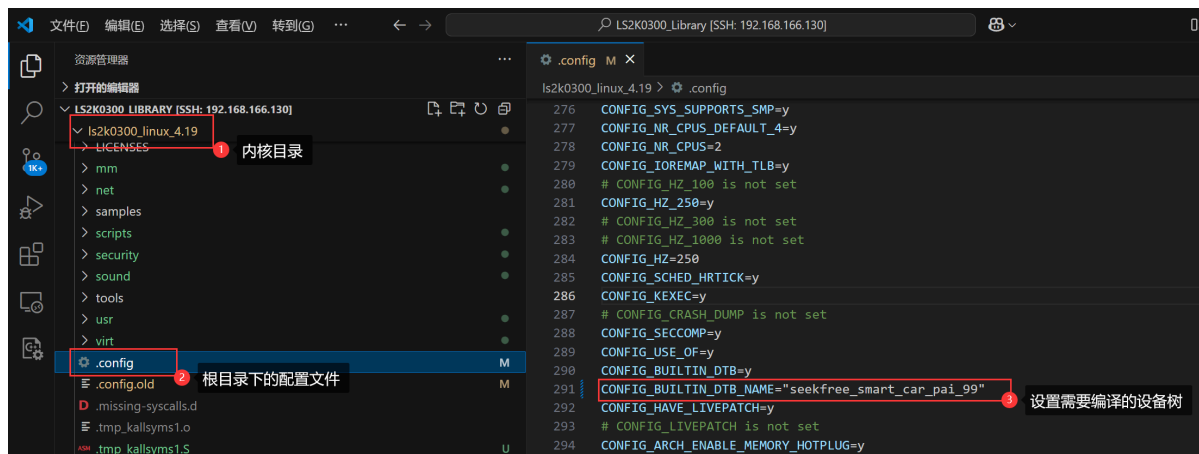
之前我们考虑过将驱动编译成模块进行加载，不过后来觉得这样会多几个不必要的操作，所以就直接把驱动编译进内核了。这样，加载内核后，我们编写的驱动也就随之加载好了。

1、配置编译哪一个设备树

1.1、打开配置文件

.config为配置文件，这个文件里面提供的编译选项。

我们找到CONFIG_DULTIN_DTB_NAME选项，可以看到，我们编译的设备树名称为：
seekfree_smart_car_pai_99



1.2、使用图形化界面打开配置文件

安装必要的软件

```
sudo apt-get install libncurses5-dev
```

在编译之前，需要先导出交叉编译工具链

```
export PATH=/opt/ls_2k0300_env/loongson-gnu-toolchain-8.3-x86_64-loongarch64-  
linux-gnu-rc1.6/bin:$PATH
```

然后输入：

```
make menuconfig
```

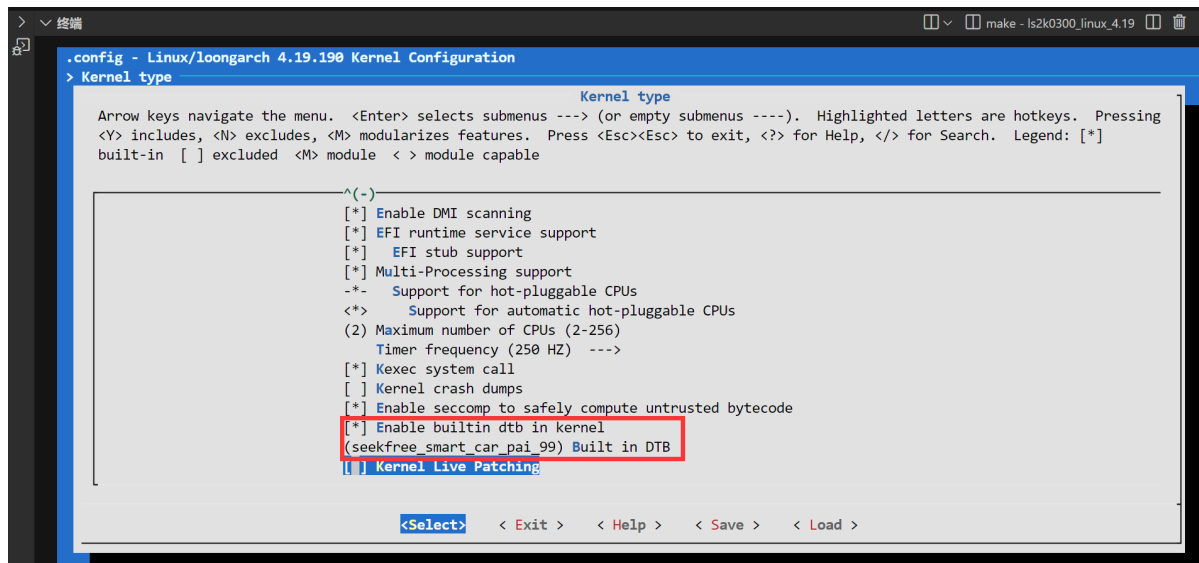
即可进入图形化配置界面。

->kernrl type

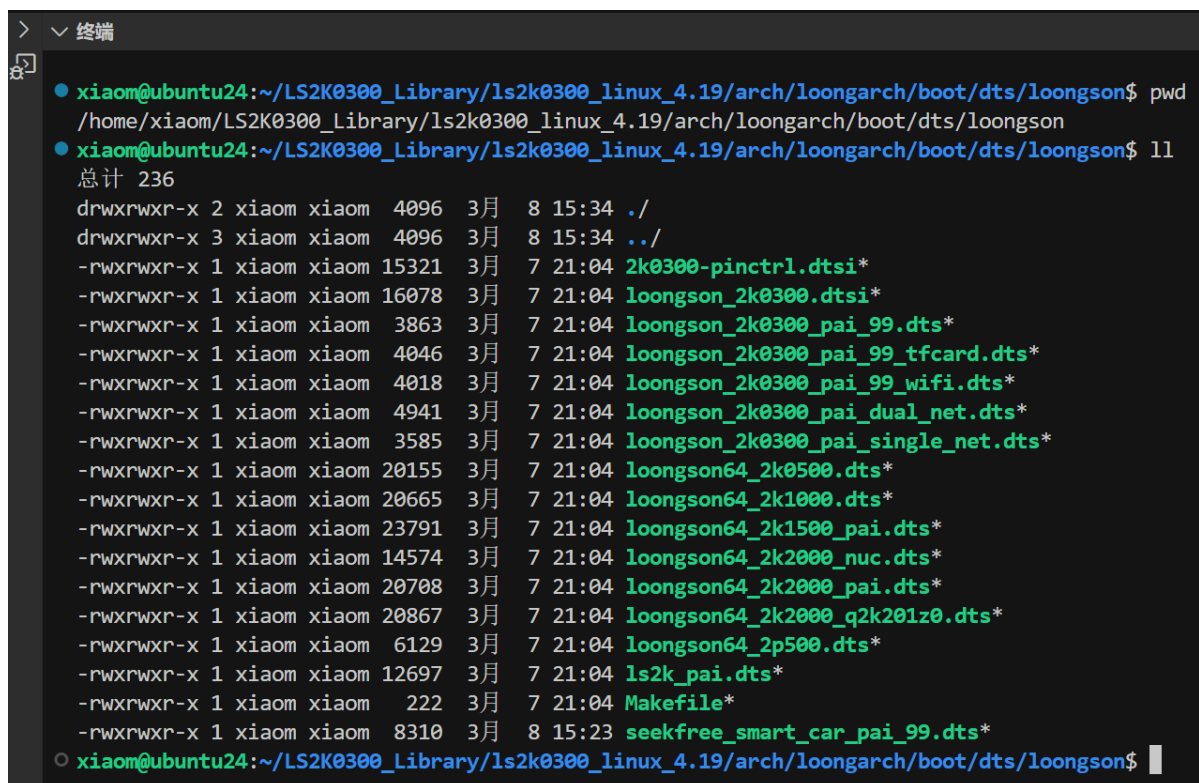
->Enable builtin dtb in kernel

->(seekfree_smart_car_pai_99) Built in DTB

按照这个路径查找，可以看到，我们的设备树名称为：seekfree_smart_car_pai_99



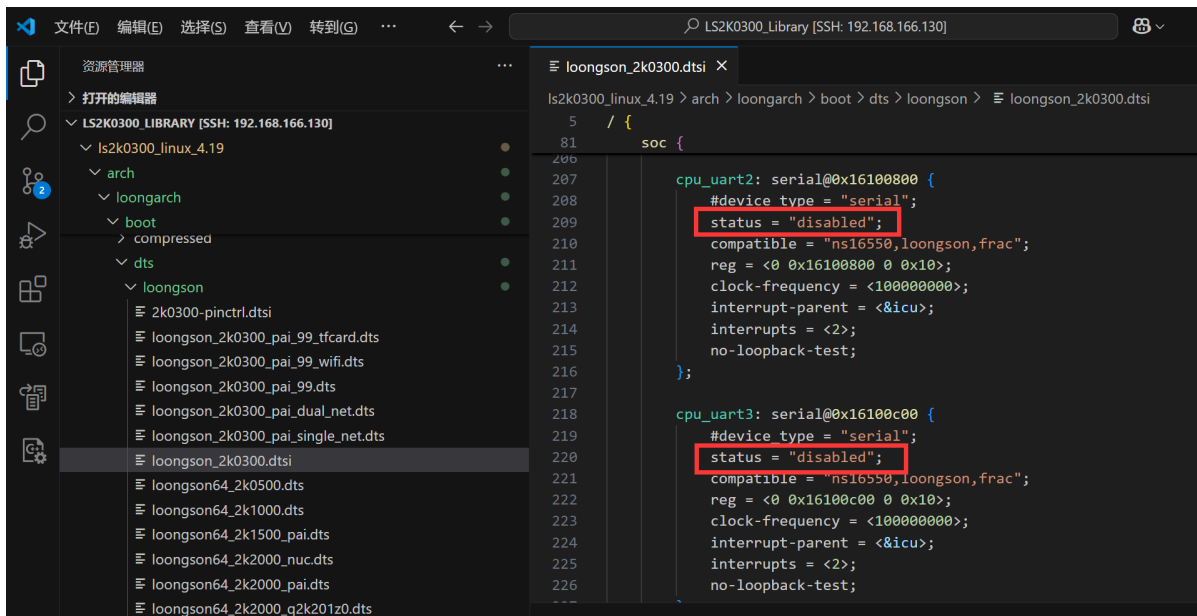
2k0300久久派板卡的设备树，存放在/arch/loongarch/boot/dts/loongson这个路径下：



这里，我们需要关注三个文件

2k0300-pinctrl.dtsi为引脚复用信息，这个文件一般是不允许修改的。

loongson_2k0300.dtsi这个文件为设备树的基础配置文件，可以打开看到，里面的status值都是"disabled"。这个文件一般也是不允许被修改的。

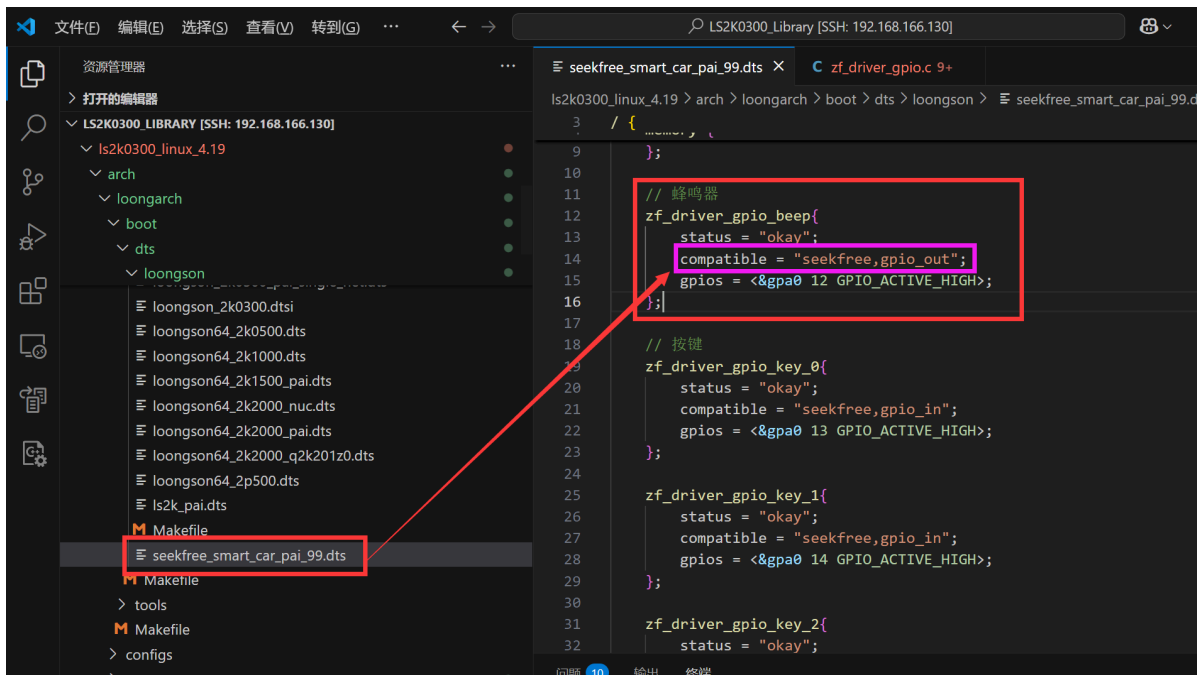


还有一个文件为seekfree_smart_car_pai_99.dts，这个文件允许用户自行配置引脚，复用等功能。

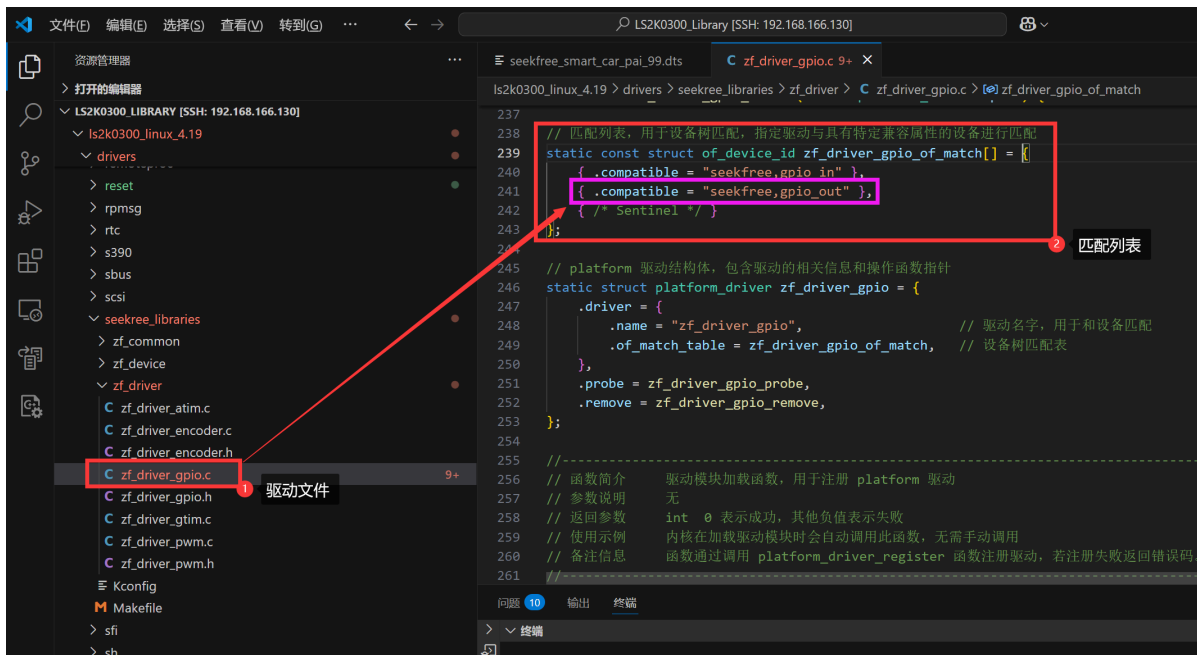
2、打开seekfree_smart_car_pai_99.dts

设备树与驱动的匹配是通过compatible属性进行匹配。接下来，我们简单讲解下设备树。

我们以蜂鸣器属性为例：



compatible 属性为: "seekfree,gpio_out"



设备树中和驱动中，有相同的compatible属性，就代表这个设备树，使用该驱动文件。

2.1、compatible = "seekfree,gpio_out"

蜂鸣器和8701E驱动的方向引脚，需要设置为输出。

```
zf_driver_gpio_beep{
    status = "okay";
    compatible = "seekfree,gpio_out";
    gpios = <&gpa0 12 GPIO_ACTIVE_HIGH>;
};
```

我们以蜂鸣器为例：

1. `zf_driver_gpio_beep` 为设备的名称，会在久久派板卡中生成 `/dev/zf_driver_gpio_beep` 这个文件，我们只需要写这个文件，就能实现蜂鸣器的控制。
2. `status` 表示设置设备的状态。"okay" 表示该设备处于可用状态，当 Linux 内核解析设备树时，会识别到这个设备并尝试对其进行初始化和配置。如果状态设置为 "disabled"，则内核会忽略该设备。
3. `compatible` 的属性为匹配字符串，只有设备树与驱动匹配才会自动进行加载。
4. `gpios` 为引脚，`gpa0` 代表第0组，每一组引脚有16个，引脚号为12，也就是GPIO12引脚，引脚的计算公式为： $0 \times 16 + 12 = 12$

2.2、compatible = "seekfree,gpio_in"

按键和拨码开关等引脚，需要设置为输入。

```
zf_driver_gpio_key_3{
    status = "okay";
    compatible = "seekfree,gpio_in";
    gpios = <&gpa1 0 GPIO_ACTIVE_HIGH>;
};
```

以 `zf_driver_gpio_key_3` 为例：

1. `zf_driver_gpio_key_3` 为设备的名称，会在久久派板卡中生成 `/dev/zf_driver_gpio_key_3` 这个文件，我们只需要读这个文件，就能可以获取引脚状态。
2. `status` 表示设置设备的状态。"okay" 表示该设备处于可用状态，当 Linux 内核解析设备树时，会识别到这个设备并尝试对其进行初始化和配置。如果状态设置为 "disabled"，则内核会忽略该设备。
3. `compatible` 的属性为匹配字符串，只有设备树与驱动匹配才会自动进行加载。
4. `gpios` 为引脚，`gpa1` 代表第1组，每一组引脚有16个，引脚号为0，也就是GPIO16引脚，引脚的计算公式为： $1 \times 16 + 0 = 16$

2.3、compatible = "seekfree,imu"

使用SPI驱动的IMU660RA、IMU660RB、IMU963RA。这个驱动会自动判断是什么设备，然后自动生成IIO设备。

可以在 `/sys/bus/iio/devices/iio:device1/` 目录下看到，六轴或者九轴的设备文件。

这段设备树代码描述了一个基于 GPIO 模拟的 SPI 总线控制器 `spi3`，并在该总线上挂载了一个 IMU 设备 `imu`。通过这些信息，Linux 内核可以识别和配置这两个设备，实现相应的通信和控制功能。

```
spi3 {
    status = "okay";
    compatible = "spi-gpio";

    gpio-sck = <&gpa1 6 GPIO_ACTIVE_HIGH>;    // GPIO22
    gpio-mosi = <&gpa1 7 GPIO_ACTIVE_HIGH>;    // GPIO23
    gpio-miso = <&gpa1 8 GPIO_ACTIVE_HIGH>;    // GPIO24
    cs-gpios = <&gpa1 9 GPIO_ACTIVE_HIGH>;    // GPIO25

    #address-cells = <1>;
    #size-cells = <0>;
    num-chipselects = <1>;

    imu: imu@0 {
        status = "okay";
        gpio-controller;
        compatible = "seekfree,imu";    // 自动识别IMU660RA IMU660RB IMU963RA
        spi-max-frequency = <10000000>;
        reg = <0>;
    };
};
```

1. spi3节点

```
spi3 {
    status = "okay";
    compatible = "spi-gpio";
```

- `status` 表示设置设备的状态。"okay" 表示该设备处于可用状态，当 Linux 内核解析设备树时，会识别到这个设备并尝试对其进行初始化和配置。如果状态设置为 "disabled"，则内核会忽略该设备。
- `compatible = "spi-gpio"`: 表明该设备与 `spi-gpio` 兼容，即它是一个基于 GPIO (General Purpose Input Output, 通用输入输出) 模拟的 SPI 总线控制器。

```

gpio-sck = <&gpa1 6 GPIO_ACTIVE_HIGH>; // GPIO22
gpio-mosi = <&gpa1 7 GPIO_ACTIVE_HIGH>; // GPIO23
gpio-miso = <&gpa1 8 GPIO_ACTIVE_HIGH>; // GPIO24
cs-gpios = <&gpa1 9 GPIO_ACTIVE_HIGH>; // GPIO25

```

- `gpio-sck`：指定 SPI 总线的时钟线（Serial Clock）所使用的 GPIO 引脚，这里使用了 `gpa1` 组的第 6 个引脚，并且该引脚在高电平有效（`GPIO_ACTIVE_HIGH`）。注释中提到对应实际的 GPIO22。引脚的计算公式为： $1 \times 16 + 6 = 22$
- `gpio-mosi`：指定主输出从输入线（Master Out Slave In）所使用的 GPIO 引脚，对应 `gpa1` 组的第 7 个引脚，高电平有效，对应实际的 GPIO23。引脚的计算公式为： $1 \times 16 + 7 = 23$
- `gpio-miso`：指定主输入从输出线（Master In Slave Out）所使用的 GPIO 引脚，对应 `gpa1` 组的第 8 个引脚，高电平有效，对应实际的 GPIO24。引脚的计算公式为： $1 \times 16 + 8 = 24$
- `cs-gpios`：指定片选线（Chip Select）所使用的 GPIO 引脚，对应 `gpa1` 组的第 9 个引脚，高电平有效，对应实际的 GPIO25。引脚的计算公式为： $1 \times 16 + 9 = 25$

```

#address-cells = <1>;
#size-cells = <0>;
num-chipselects = <1>;

```

- `#address-cells = <1>`：表示子节点的 `reg` 属性中用于表示地址的单元格数量为 1。在设备树中，`reg` 属性用于描述设备的地址范围，`#address-cells` 定义了地址部分的长度。
- `#size-cells = <0>`：表示子节点的 `reg` 属性中用于表示大小的单元格数量为 0，即不使用大小信息。
- `num-chipselects = <1>`：表示该 SPI 总线控制器支持的片选信号数量为 1。

2. imu子节点

```

imu: imu@0 {
    status = "okay";
    gpio-controller;
    compatible = "seekfree,imu"; // 自动识别IMU660RA IMU660RB IMU963RA
    spi-max-frequency = <10000000>;
    reg = <0>;
};

```

- `imu: imu@0`：`imu` 是该节点的标签，方便在其他地方引用；`imu@0` 表示该设备在 SPI 总线上的片选编号为 0。
- `status = "okay"`：表示该 IMU 设备处于可用状态。
- `gpio-controller`：表明该设备本身可以作为一个 GPIO 控制器。
- `compatible = "seekfree,imu"`：表示该设备与 `seekfree,imu` 兼容，注释中提到可以自动识别 IMU660RA、IMU660RB 和 IMU963RA 等型号。
- `spi-max-frequency = <10000000>`：指定该 IMU 设备在 SPI 总线上支持的最大通信频率为 10MHz。
- `reg = <0>`：表示该设备在 SPI 总线上的片选编号为 0，与节点名称中的 `@0` 对应。

2.4、compatible = "sitronix,st7789v"

这个节点为IPS200屏幕，目前只支持IPS200屏幕。

即使同样驱动的IPS114，因分辨率和坐标偏移是不一样的。所以目前不支持。

```
&spi1{
    status = "okay";
    pinctrl-names = "default";
    pinctrl-0 = <&spi1_4bit>;
    st7789v:st7789v@0{
        status = "okay";
        compatible = "sitronix,st7789v";
        reg = <0>;                //chip select 0:cs0  1:cs1
        spi-max-frequency = <50000000>;
        spi-cpol;
        spi-cpha;
        rgb;
        rotate = <0>;              // 旋转方向 0 90 180 270
        buswidth = <8>;
        fps = <30>;                // 帧率
        dc-gpios = <&gpa1 10 GPIO_ACTIVE_HIGH>;    // GPIO26
        reset-gpios = <&gpa4 10 GPIO_ACTIVE_HIGH>; // GPIO74
        width = <240>;
        height = <320>;
        debug = <0x0>;
    };
};
```

这个节点是使用的硬件SPI驱动屏幕。

节点内容按照格式填写即可。

2.5、compatible = "seekfree,encoder"

在龙芯久久派板卡上面有四个独立的PWM模块，每一个模块只有一个通道。

而编码器的采集也是使用的PWM模块，所以，这里的设备树，我们直接引用PWM模块。重新设置compatible 匹配属性，匹配我们自己写的驱动。

```
&pwm0{
    status = "okay";
    pinctrl-names = "default";
    pinctrl-0 = <&pwm0_mux_m0>;
    clock-frequency = <100000000>;
    compatible = "seekfree,encoder";
    encoder-dir-gpio = <&gpa3 3 GPIO_ACTIVE_HIGH>;
    dev-name = "zf_encoder_1";
};
&pwm3{
    status = "okay";
    pinctrl-names = "default";
    pinctrl-0 = <&pwm3_mux_m0>;
    clock-frequency = <100000000>;
    reg = <0 0x1611b030 0 0x10>;
};
```

```
compatible = "seekfree,encoder";
encoder-dir-gpio = <&gpa3 2 GPIO_ACTIVE_HIGH>;
dev-name = "zf_encoder_2";
};
```

我们以pwm0为例：

1. `status` 表示设置设备的状态。"okay" 表示该设备处于可用状态，当 Linux 内核解析设备树时，会识别到这个设备并尝试对其进行初始化和配置。如果状态设置为 "disabled"，则内核会忽略该设备。
2. `pinctrl-names` 属性定义了一组引脚控制状态的名称。这里设置为 "default"，意味着该设备使用名为 "default" 的引脚控制配置。引脚控制配置通常用于指定设备所使用的 GPIO 引脚的复用功能、电气特性等。
3. `clock-frequency` 属性设置设备的时钟频率，这里将时钟频率设置为 100000000Hz（即 100MHz）。编码器设备通常需要一个稳定的时钟信号来正常工作，这个时钟频率会影响编码器的采样速度和数据处理能力。
4. `pinctrl-0` 属性指定了与 `pinctrl-names` 中第一个名称（这里是 "default"）对应的引脚控制配置节点。<&pwm0_mux_m0> 是一个引用，指向设备树中名为 `pwm0_mux_m0` 的节点，该节点应该定义了该设备使用的引脚的具体配置信息，比如引脚的复用模式、上拉 / 下拉电阻等。
5. 这行代码设置设备的状态。"okay" 表示该设备处于可用状态，当 Linux 内核解析设备树时，会识别到这个设备并尝试对其进行初始化和配置。如果状态设置为 "disabled"，则内核会忽略该设备。
6. `compatible` 的属性为匹配字符串，只有设备树与驱动匹配才会自动进行加载。
7. `encoder-dir-gpio` 属性指定了编码器的方向控制引脚。<&gpa3 3 GPIO_ACTIVE_HIGH> 表示使用 `gpa3` 这个 GPIO 控制器的第 3 个引脚，并且该引脚在高电平（GPIO_ACTIVE_HIGH）时有效。编码器的方向控制引脚通常用于控制编码器的计数方向（例如正向计数或反向计数）。
8. `dev-name` 属性为设备指定一个名称，这里将设备命名为 `zf_encoder_1`。这个会在久久派板卡中生成 `/dev/zf_encoder_1` 这个文件，我们只需要读这个文件，就能获取编码器的数据。

2.6、compatible = "loongson,ls300-pwm"

```
&pwm1{
    status = "okay";
    pinctrl-names = "default";
    pinctrl-0 = <&pwm1_mux_m0>;
    clock-frequency = <100000000>;
    compatible = "loongson,ls300-pwm";
};
&pwm2{
    status = "okay";
    pinctrl-names = "default";
    pinctrl-0 = <&pwm2_mux_m0>;
    clock-frequency = <100000000>;
    compatible = "loongson,ls300-pwm";
};
```

我们以pwm1为例：

1. `status` 表示设置设备的状态。"okay" 表示该设备处于可用状态，当 Linux 内核解析设备树时，会识别到这个设备并尝试对其进行初始化和配置。如果状态设置为 "disabled"，则内核会忽略该设备。
2. `pinctrl-names` 属性定义了一组引脚控制状态的名称。这里设置为 "default"，意味着该设备使用名为 "default" 的引脚控制配置。引脚控制配置通常用于指定设备所使用的 GPIO 引脚的复用功能、电气特性等。
3. `pinctrl-0` 属性指定了与 `pinctrl-names` 中第一个名称（这里是 "default"）对应的引脚控制配置节点。`<&pwm0_mux_m0>` 是一个引用，指向设备树中名为 `pwm0_mux_m0` 的节点，该节点应该定义了该设备使用的引脚的具体配置信息，比如引脚的复用模式、上拉 / 下拉电阻等。
4. `clock-frequency` 属性设置设备的时钟频率，这里将时钟频率设置为 100000000Hz（即 100MHz）。编码器设备通常需要一个稳定的时钟信号来正常工作，这个时钟频率会影响编码器的采样速度和数据处理能力。
5. `compatible` 的属性为匹配字符串，只有设备树与驱动匹配才会自动进行加载。

2.7、compatible = "loongson,ls300-pwm-atim"

```
&pwm_atim{
    status = "okay";
    compatible = "loongson,ls300-pwm-atim";
    pinctrl-names = "default";
    pinctrl-0 = <&atim_ch3n_pin_m1>;
};
```

1. `status` 表示设置设备的状态。"okay" 表示该设备处于可用状态，当 Linux 内核解析设备树时，会识别到这个设备并尝试对其进行初始化和配置。如果状态设置为 "disabled"，则内核会忽略该设备。
2. `compatible` 的属性为匹配字符串，只有设备树与驱动匹配才会自动进行加载。
3. `pinctrl-names` 属性定义了一组引脚控制状态的名称。这里设置为 "default"，意味着该设备使用名为 "default" 的引脚控制配置。引脚控制配置通常用于指定设备所使用的 GPIO 引脚的复用功能、电气特性等。
4. `pinctrl-0` 属性指定了与 `pinctrl-names` 中第一个名称（这里是 "default"）对应的引脚控制配置节点。`<&atim_ch3n_pin_m1>` 是一个引用，指向设备树中名为 `atim_ch3n_pin_m1` 的节点，该节点应该定义了该设备使用的引脚的具体配置信息，比如引脚的复用模式、上拉 / 下拉电阻等。

2.8、compatible = "loongson,ls300-pwm-gtim"

```
&pwm_gtim{
    status = "okay";
    compatible = "loongson,ls300-pwm-gtim";
    pinctrl-names = "default";
    pinctrl-0 = <&gtgtim_ch2_pin_m1>, <&gtgtim_ch3_pin_m1>;
};
```

1. `status` 表示设置设备的状态。"okay" 表示该设备处于可用状态，当 Linux 内核解析设备树时，会识别到这个设备并尝试对其进行初始化和配置。如果状态设置为 "disabled"，则内核会忽略该设备。
2. `compatible` 的属性为匹配字符串，只有设备树与驱动匹配才会自动进行加载。

3. `pinctrl-names` 属性定义了一组引脚控制状态的名称。这里设置为 `"default"`，意味着该设备使用名为 `"default"` 的引脚控制配置。引脚控制配置通常用于指定设备所使用的 GPIO 引脚的复用功能、电气特性等。
4. `pinctrl-0` 属性指定了与 `pinctrl-names` 中第一个名称（这里是 `"default"`）对应的引脚控制配置节点。`<atim_ch3n_pin_m1>` 是一个引用，指向设备树中名为 `atim_ch3n_pin_m1` 的节点，该节点应该定义了该设备使用的引脚的具体配置信息，比如引脚的复用模式、上拉 / 下拉电阻等。

2.9、compatible = "seekfree,pwm"

舵机、电机、电调等设备，都需要设置为pwm。

```
zf_device_pwm_servo{
    compatible = "seekfree,pwm";
    pwms = <pwm_atim 5 1000000>;    // pwm_atim 通道5 周期(用户不需要关心) (0-6 一共7
    个通道)
    status = "okay";
    freq = <300>;                    // 300HZ 频率
    duty = <0>;                      // 默认 0% 的占空比
    duty_max = <10000>;              // duty最大值，不建议修改。
};
```

这里我们以舵机PWM为例：

1. `zf_device_pwm_servo` 为设备的名称，会在久久派板卡中生成 `/dev/zf_device_pwm_servo` 这个文件，我们只需要写这个文件，就能可以修改PWM的duty值。
2. `compatible` 的属性为匹配字符串，只有设备树与驱动匹配才会自动进行加载。
3. `pwms` 属性用于指定 PWM 信号的相关信息。
 - `&pwm_atim`：这是一个引用，指向另一个设备树节点 `pwm_atim`，表示使用这个节点所代表的 PWM 控制器。
 - `5`：表示使用 `pwm_atim` 控制器的第 5 个通道来生成 PWM 信号。有 0 - 6 共 7 个通道可供选择。
 - `1000000`：代表 PWM 信号的周期，用户不需要关心这个值。
4. `status` 表示设置设备的状态。`"okay"` 表示该设备处于可用状态，当 Linux 内核解析设备树时，会识别到这个设备并尝试对其进行初始化和配置。如果状态设置为 `"disabled"`，则内核会忽略该设备。
5. `freq` 属性指定了 PWM 信号的频率，这里设置为 300Hz。舵机通常需要特定频率的 PWM 信号来控制其转动角度。
6. `duty` 属性表示 PWM 信号初始化的占空比。
7. `duty_max` 属性定义了占空比的最大值，建议不要修改这个值。